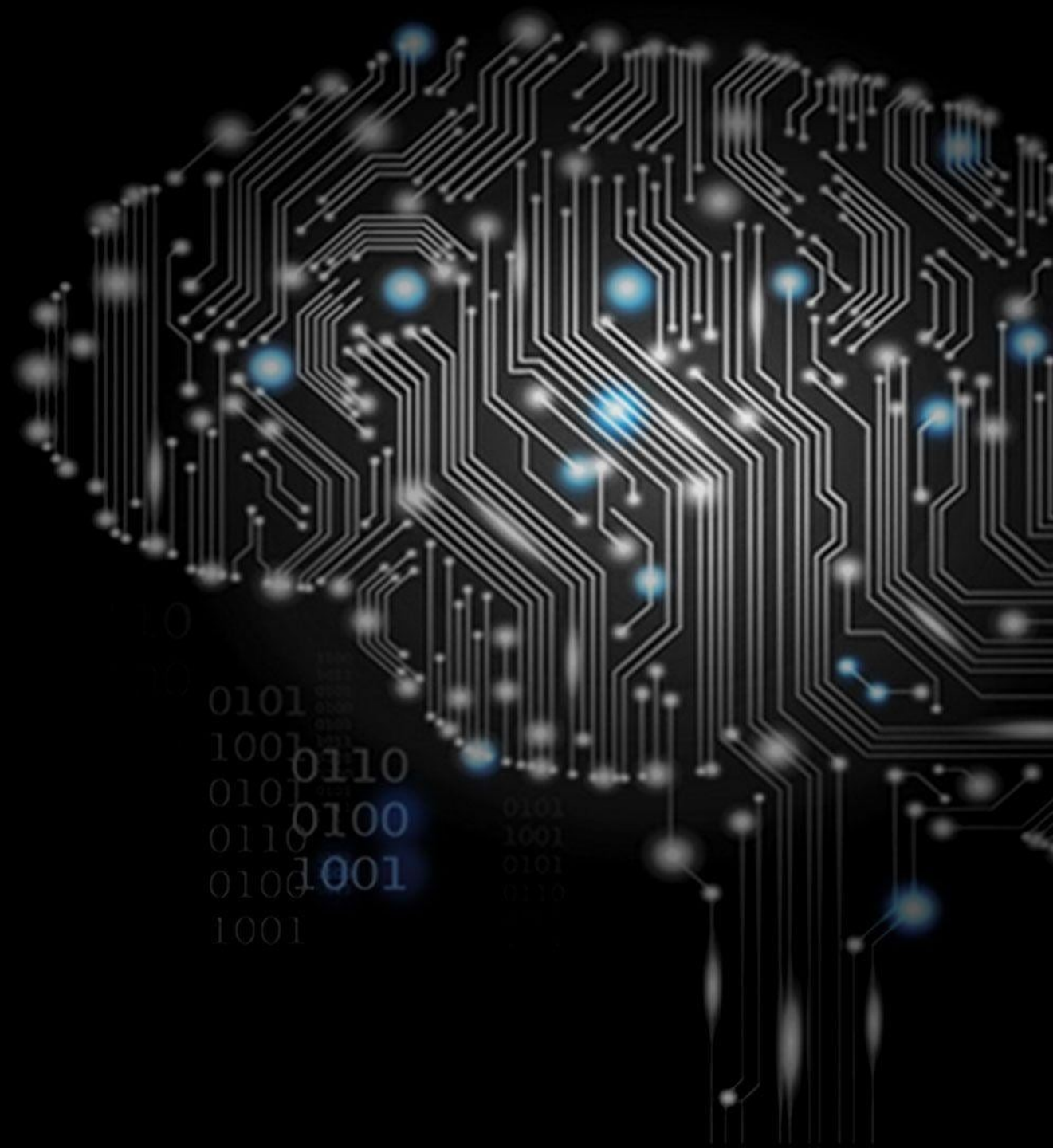


Introduction to Generative AI

5. Retrieval-Augmented Generation (RAG)

Module Topics

- LLM Limitations
- RAG Concepts
- RAG Architecture
- Chunking and Embedding
- Vector Databases and Retrieval
- Vector Database Products
- Prompt Construction
- RAG System Evaluation

© 2026 by Innovation In Software Corporation

LLM Limitations

- Hallucinations
  - LLMs generate statistically plausible text which are no necessarily concrete facts
  - They may convincingly fabricate
    - Policies that don’t exist
    - Non-existent regulations
    - Incorrect internal procedures
- Outdated training data
  - Models are trained on static datasets.
    - Don’t have access to the latest documents
    - Don’t have access to current data
    - Don’t have access to internal knowledge bases

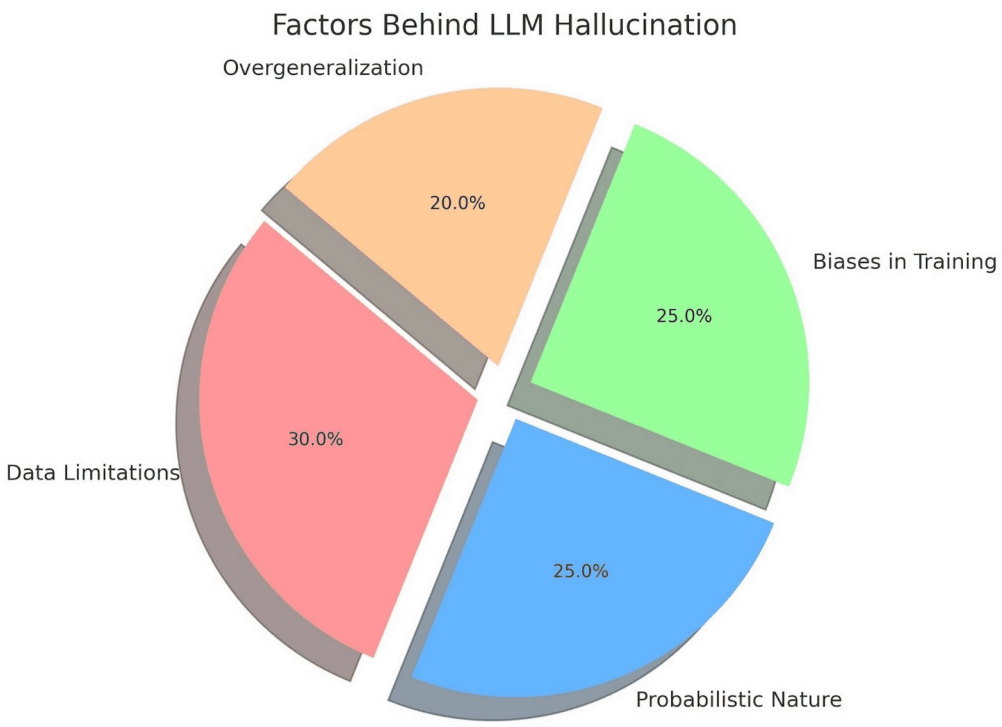


Image Credit: https://futureagi.com/blogs/understanding-llm-hallucination-2025

© 2026 by Innovation In Software Corporation

LLM Limitations

- Lack of domain specific knowledge
  - General-purpose LLMs
    - Don’t understand proprietary or industry specific terminology
    - Don’t know internal systems that are unique to an industry, organization or department
    - Cannot access private documents that they have not been trained on
- Fine tuning can mitigate this to a degree

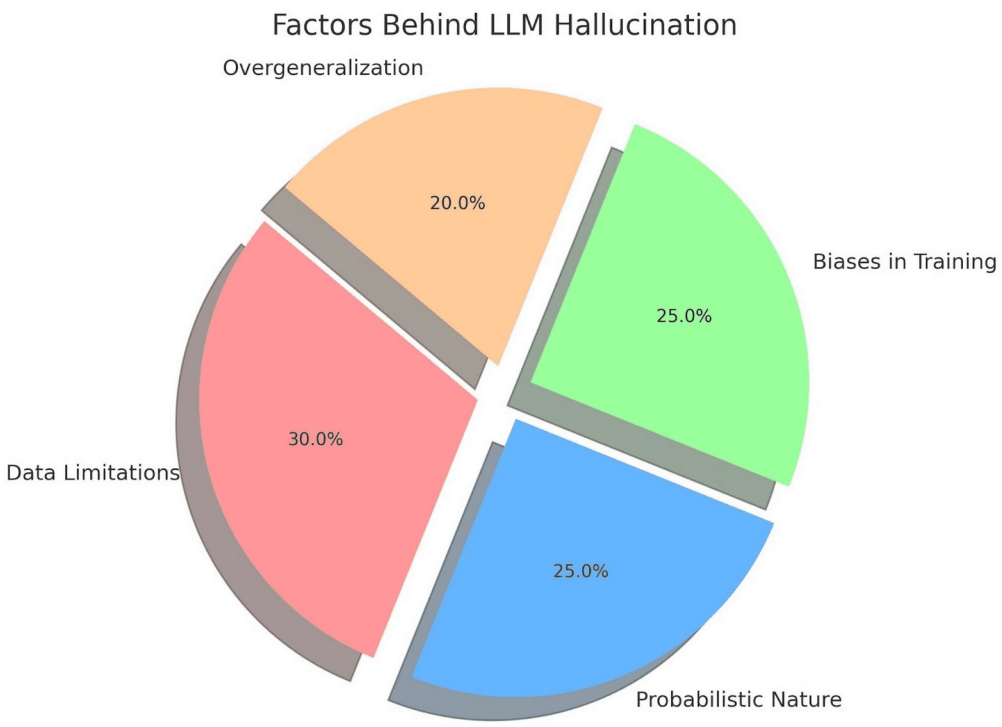
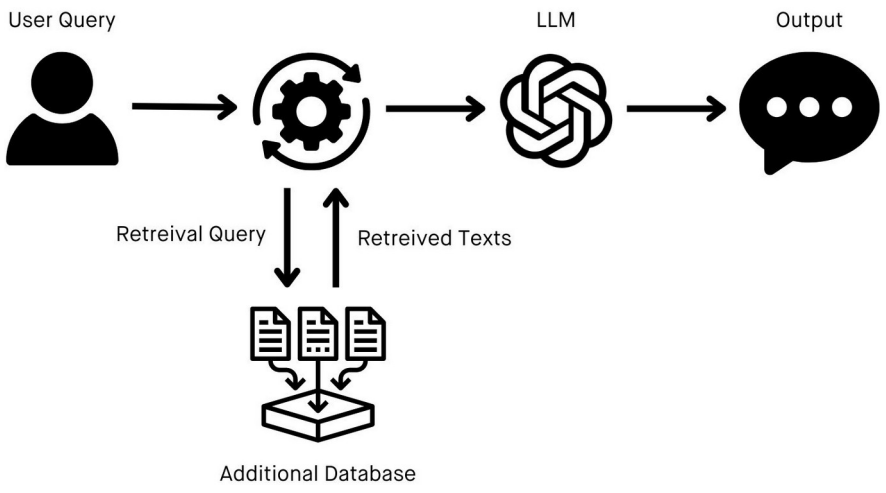


Image Credit: https://futureagi.com/blogs/understanding-llm-hallucination-2025

© 2026 by Innovation In Software Corporation

RAG Concepts

- Basic concept
  - The LLM has parametric information
    - Specifically the weights in the model
    - To update this requires retraining
  - Incorporate non-parametric data into the query itself
  - Like a super enriched form of in-context learning
  - Retrieved documents are inserted into the model's context window
    - Model conditions on them during generation
    - No weight updates occur
    - The model adapts responses based purely on context tokens

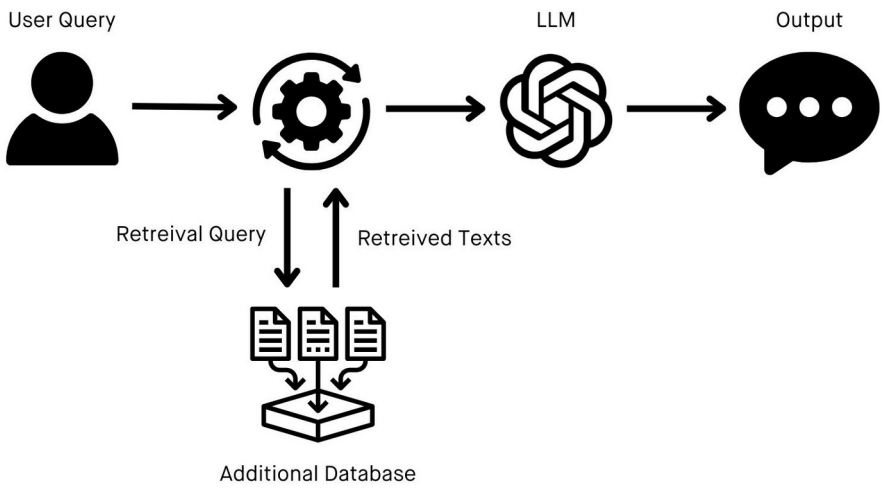


© 2026 by Innovation In Software Corporation

Image Credit: https://www.deptagency.com/en-uki/insight/an-intro-to-rag-the-best-first-step-in-your-ai-transformation/

RAG Concepts

- RAG grounds LLM outputs in external data
- Instead of relying only on its training data, the model
  - Retrieves relevant documents
  - Injects them into the prompt
  - Generates answers based on retrieved context
- Dramatically reduces hallucinations

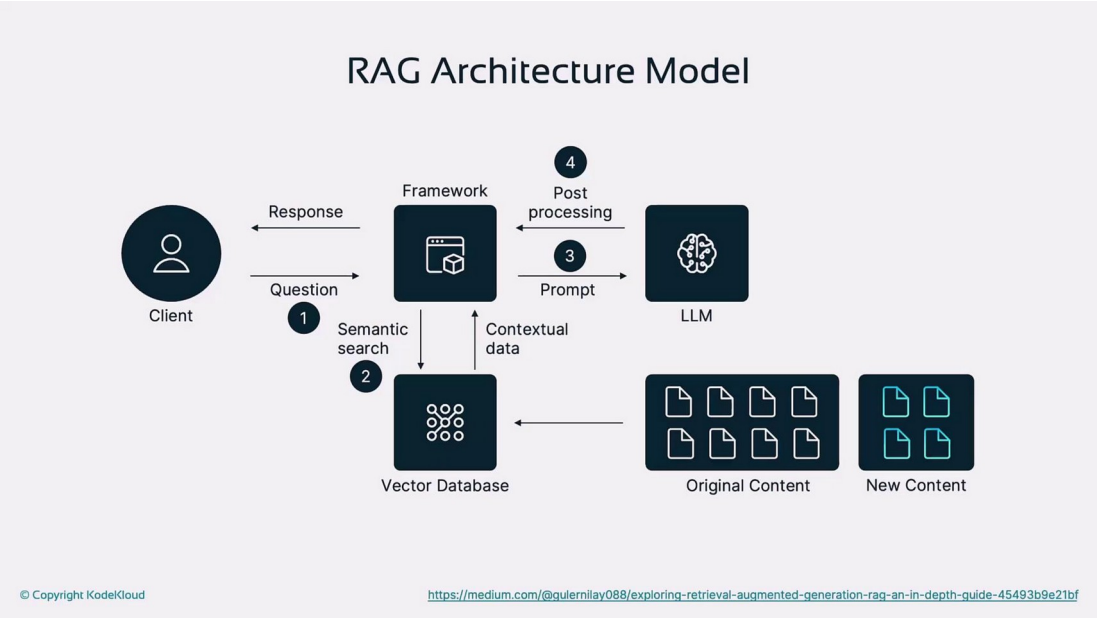


© 2026 by Innovation In Software Corporation

Image Credit: https://www.deptagency.com/en-uki/insight/an-intro-to-rag-the-best-first-step-in-your-ai-transformation/

RAG Architecture

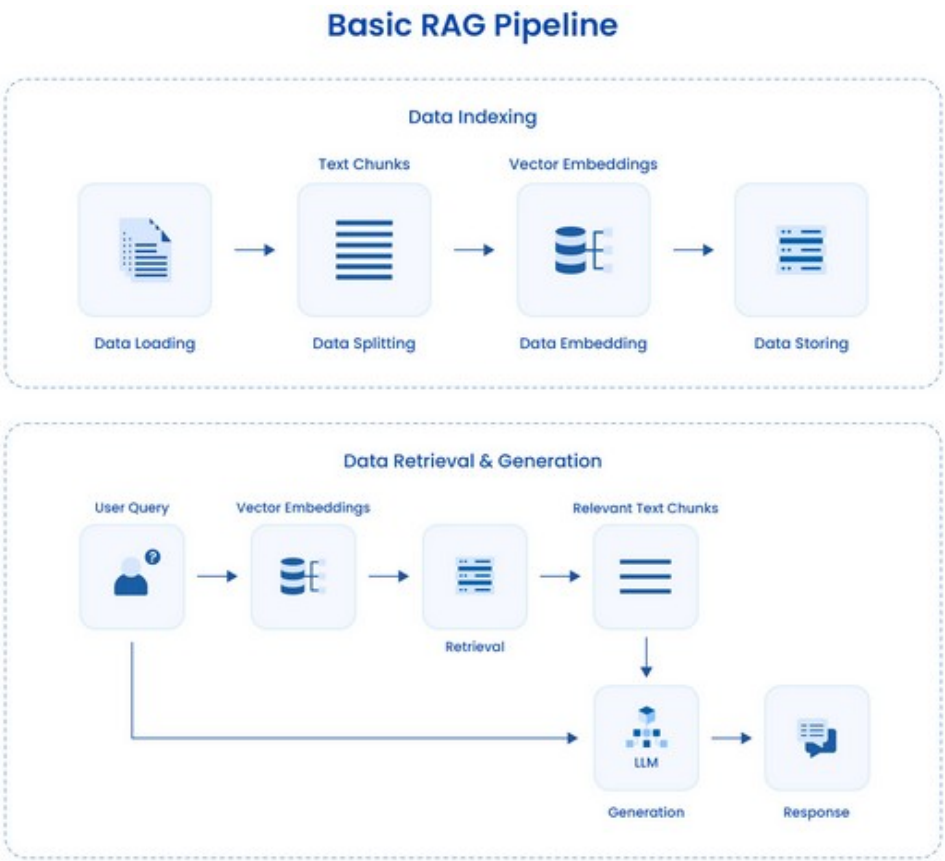
- RAG is a system pipeline
  - Document ingestion
  - Chunking
  - Embedding generation
  - Vector storage
  - Similarity retrieval
  - Prompt construction
  - LLM generation



© 2026 by Innovation In Software Corporation

RAG Architecture

- RAG is made up of three subsystems
- Indexing pipeline (offline)
  - Parse documents
  - Split text into chunks
  - Generate vector embeddings
  - Store in vector database
- Retrieval pipeline (online)
  - User query prompt begins events
  - Generate query embedding
  - Perform similarity search
- Generation layer
  - Inject retrieved chunks into prompt
  - Generate grounded response
- Separation is critical in production systems

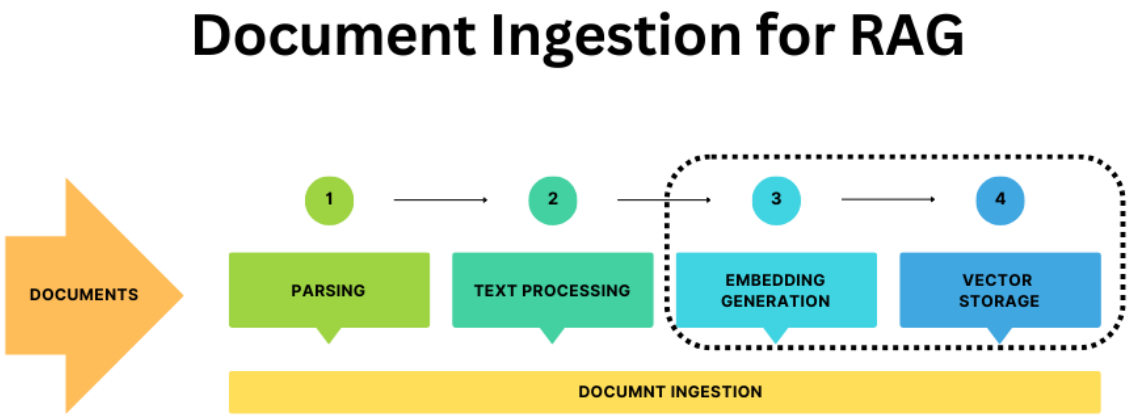


© 2026 by Innovation In Software Corporation

Image Credit: https://www.astera.com/type/blog/rag-pipeline/

The Indexing Pipeline

- Knowledge preparation layer
  - Happens offline before users ask questions
  - Essentially building a semantic search engine over the documents
- Document ingestion
  - Collects and load source documents into the system like
    - PDFs.
    - Word documents
    - HTML pages
    - Markdown files
    - Database exports
    - Emails
    - Knowledge base articles

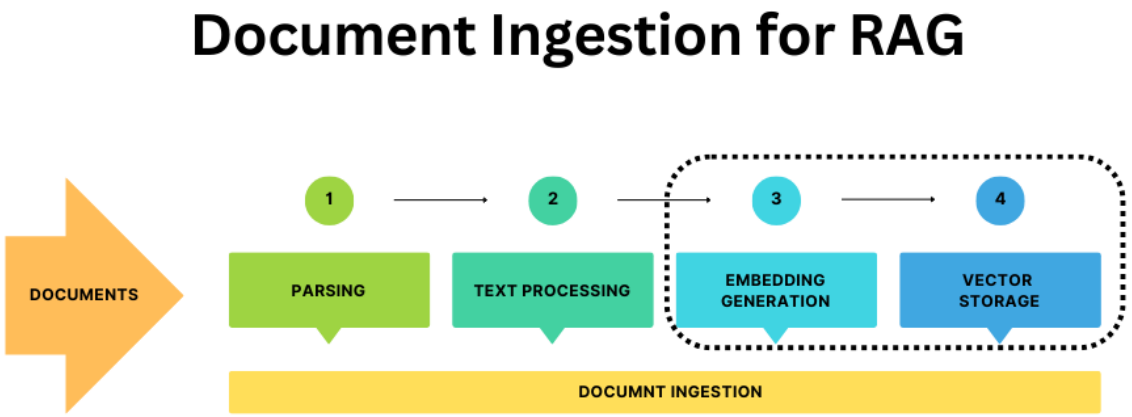


© 2026 by Innovation In Software Corporation

Image Credit: https://medium.com/@aleum.yu/document-ingestion-for-rag-parsing-and-processing-enhancing-ai-with-knowledge-ii-8c2dd8702f51

The Indexing Pipeline

- At this stage
  - The system reads raw content.
  - It extracts text.
  - It may clean up formatting
  - It may remove irrelevant elements like headers and footers
- If the source is tabular or structured
  - Converts rows into textual representations
  - Preserve metadata fields

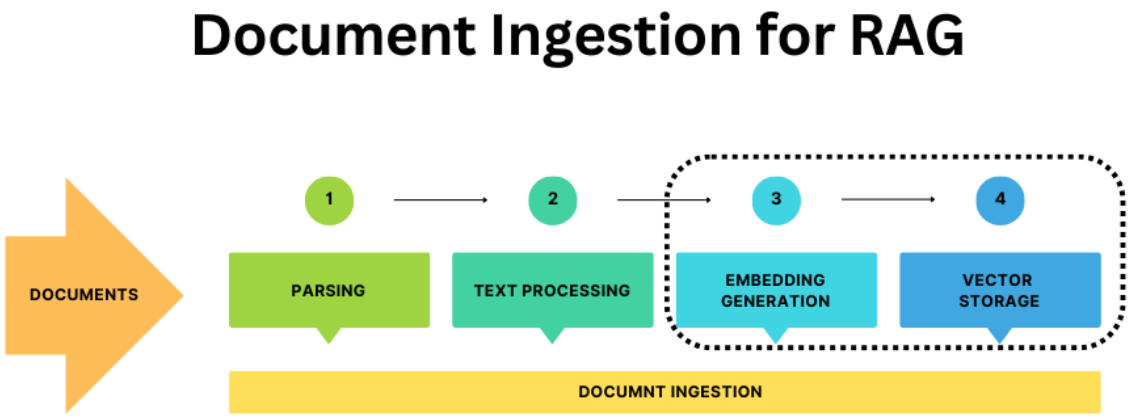


© 2026 by Innovation In Software Corporation

Image Credit: https://medium.com/@aleum.yu/document-ingestion-for-rag-parsing-and-processing-enhancing-ai-with-knowledge-ii-8c2dd8702f51

The Indexing Pipeline

- This stage is critical
  - Garbage in means we retrieve garbage
- If ingestion is not done correctly
  - Retrieval quality drops
  - Hallucinations increase
  - Context becomes noisy
- This data collection has to be curated
  - Quality measures should be used
  - As well as timeliness information
  - Follow good records management standards



© 2026 by Innovation In Software Corporation

Image Credit: https://medium.com/@aleum.yu/document-ingestion-for-rag-parsing-and-processing-enhancing-ai-with-knowledge-ii-8c2dd8702f51

Chunking

- Large documents are split into smaller pieces called chunks
  - Necessary because
    - LLMs have context limits
    - Embedding models perform better on moderate-length text
    - Retrieval works best on semantically coherent units
- Instead of embedding an entire document as one vector
  - It is split into 300–500 token segments
  - Often with overlap (e.g., 50 tokens)
  - Overlap prevents cutting off sentences or ideas

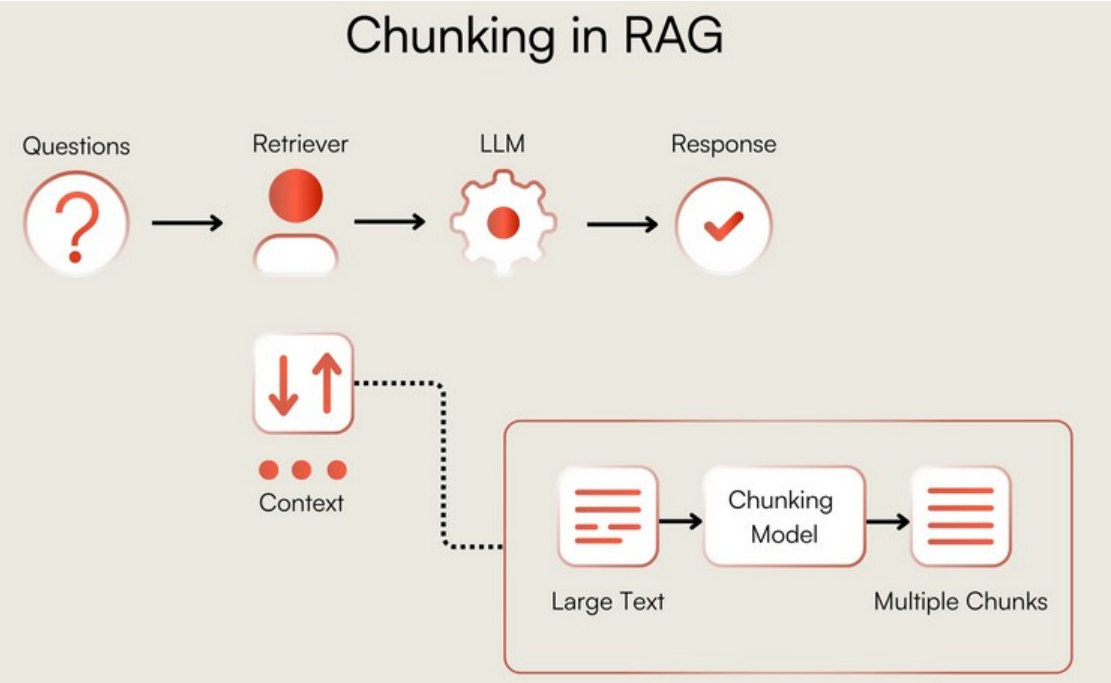


Image Credit: https://www.multimodal.dev/post/how-to-chunk-documents-for-rag

© 2026 by Innovation In Software Corporation

Chunking

- Several different ways to do chunking
- Fixed length
  - Split text every N tokens
  - Advantages
    - Deterministic
    - Fast
    - Easy to implement
    - Works reasonably well in many cases
  - Drawbacks
    - May cut sentences in half
    - May split logical sections
    - Ignores document structure

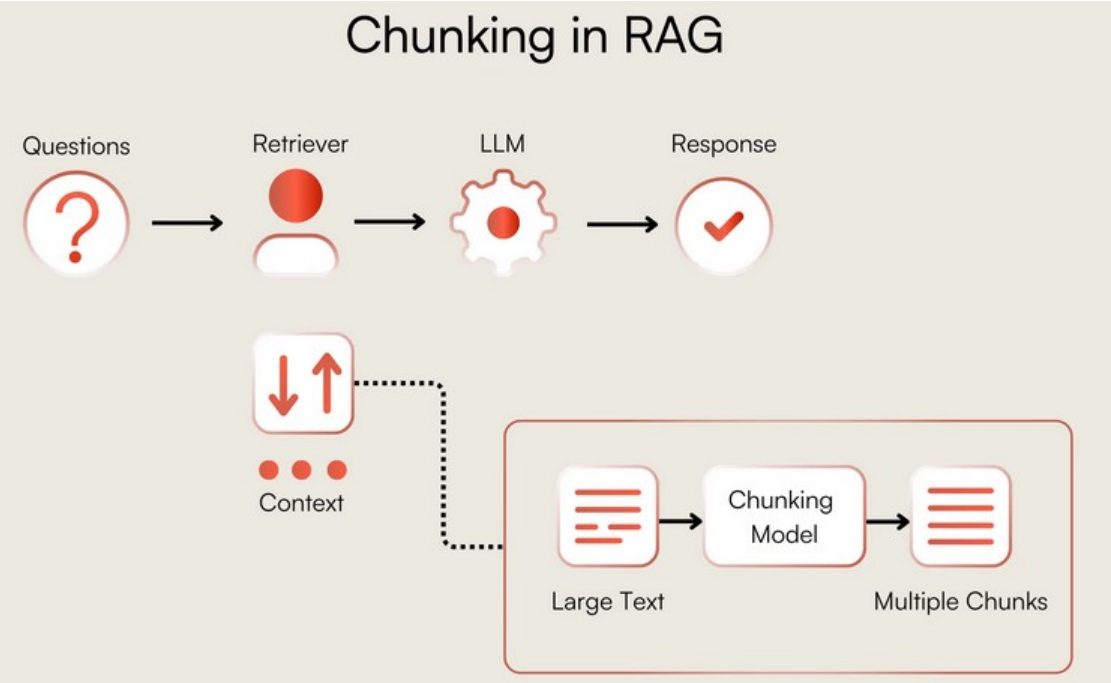


Image Credit: https://www.multimodal.dev/post/how-to-chunk-documents-for-rag

© 2026 by Innovation In Software Corporation

Chunking

- Sentence based
  - Split document into sentences
  - Group sentences until token limit is reached
  - Advantages
    - Preserves semantic coherence
    - Avoids mid-sentence splits
    - Better embedding quality
  - Drawbacks
    - Sentence length varies widely
    - Some chunks become uneven
    - Doesn't respect higher-level structure

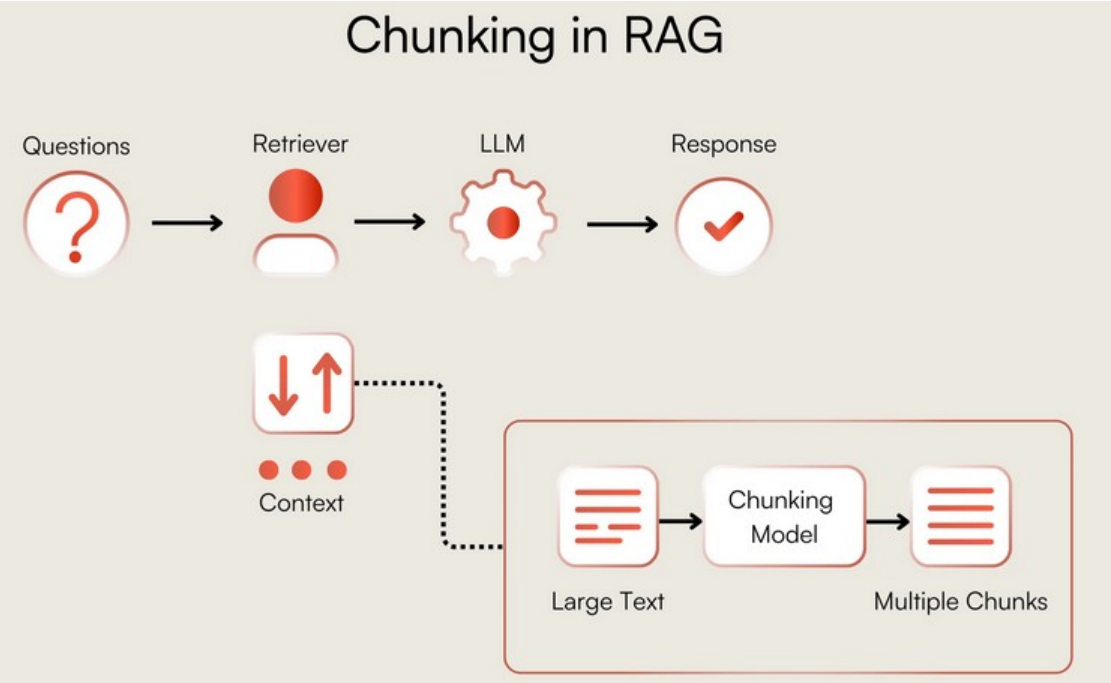


Image Credit: https://www.multimodal.dev/post/how-to-chunk-documents-for-rag

© 2026 by Innovation In Software Corporation

Chunking

- Paragraph based
  - Uses document formatting
  - Split by paragraph breaks
  - Merge paragraphs until token limit reached
- Works well when
  - Documents are well-structured
  - Paragraphs correspond to semantic units
- Fails when
  - Formatting is inconsistent (e.g., PDFs)

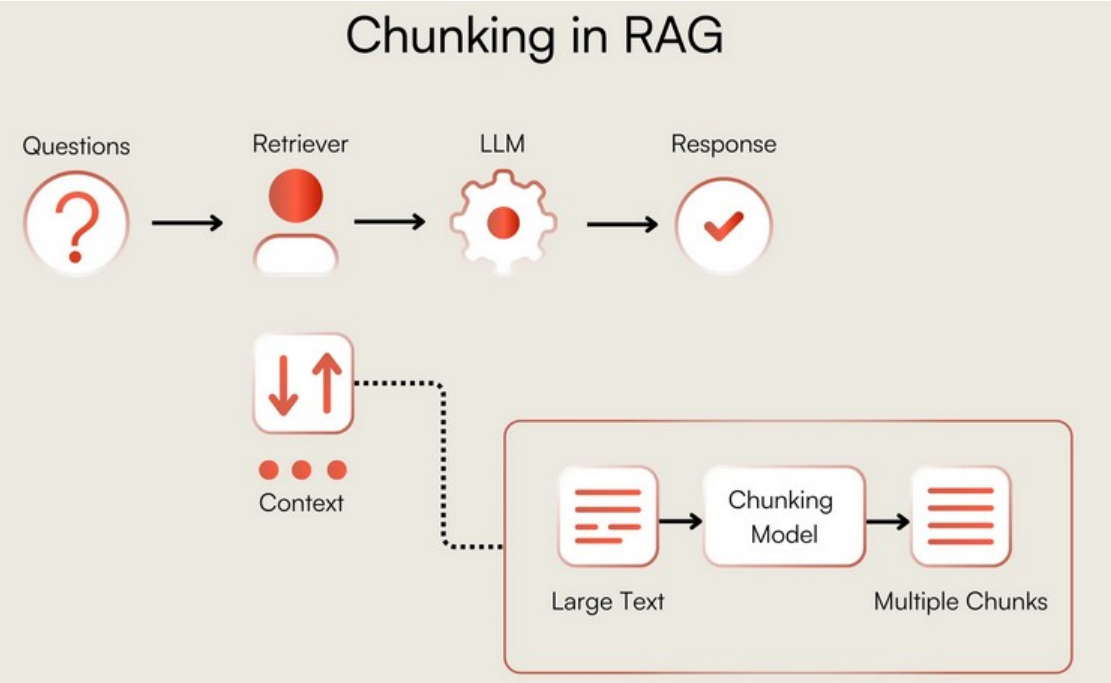


Image Credit: https://www.multimodal.dev/post/how-to-chunk-documents-for-rag

© 2026 by Innovation In Software Corporation

Chunking

- Recursive / Hierarchical
  - Common in advanced RAG systems.
  - Algorithm strategy
    - Try splitting by sections
    - If section too large then split by paragraphs
    - If paragraph too large then split by sentences
    - If sentence too large then split by tokens
  - Often superior to fixed-length splitting
    - Preserves semantic hierarchy
    - Maintains topical grouping
    - Improves retrieval relevance

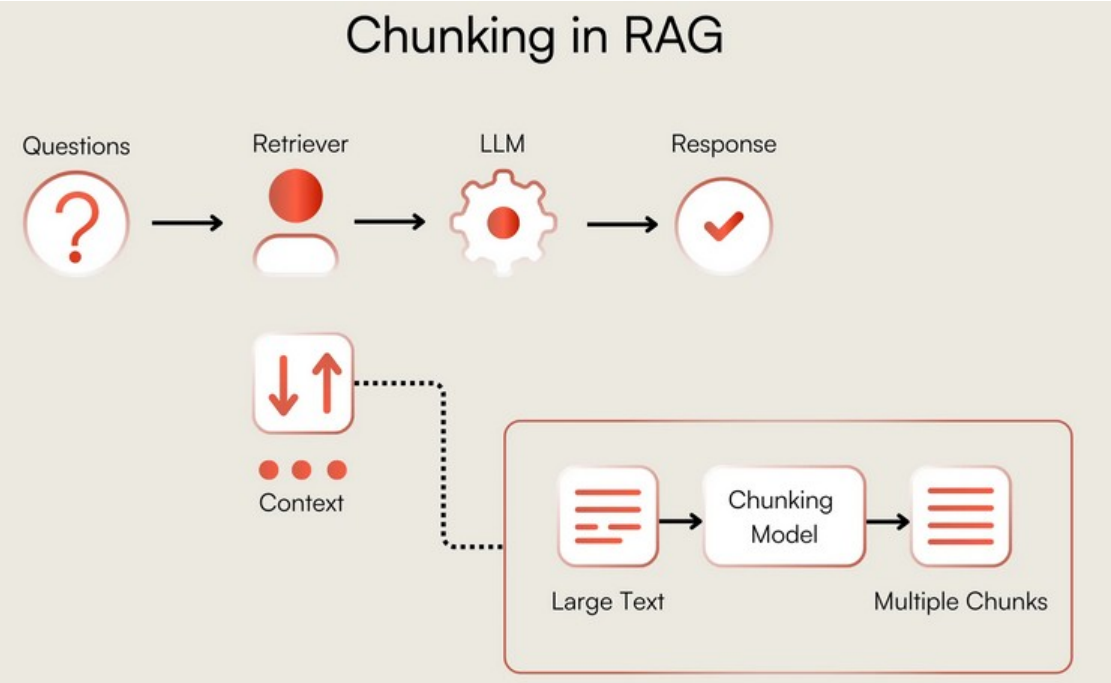


Image Credit: https://www.multimodal.dev/post/how-to-chunk-documents-for-rag

© 2026 by Innovation In Software Corporation

Chunking

- Semantic
  - Split based on semantic change.
  - Algorithm
    - Embed sentences
    - Compute similarity between adjacent sentences
    - If similarity drops below threshold then start a new chunk
  - Creates semantically coherent segments

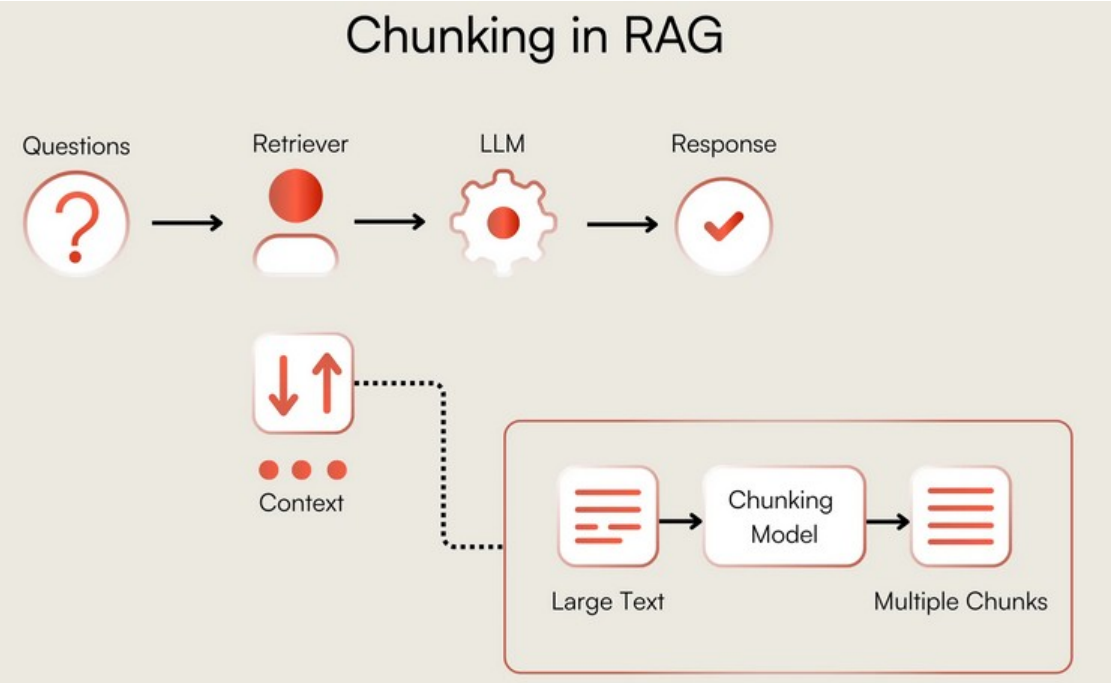


Image Credit: https://www.multimodal.dev/post/how-to-chunk-documents-for-rag

© 2026 by Innovation In Software Corporation

Chunking

- Structure-aware
  - Used for
    - Legal documents
    - Technical manuals
    - Code repositories
  - Approach
    - Preserve headers
    - Maintain section IDs
    - Store hierarchical metadata
  - Enables
    - Better filtering
    - Section-level retrieval
    - Citation clarity

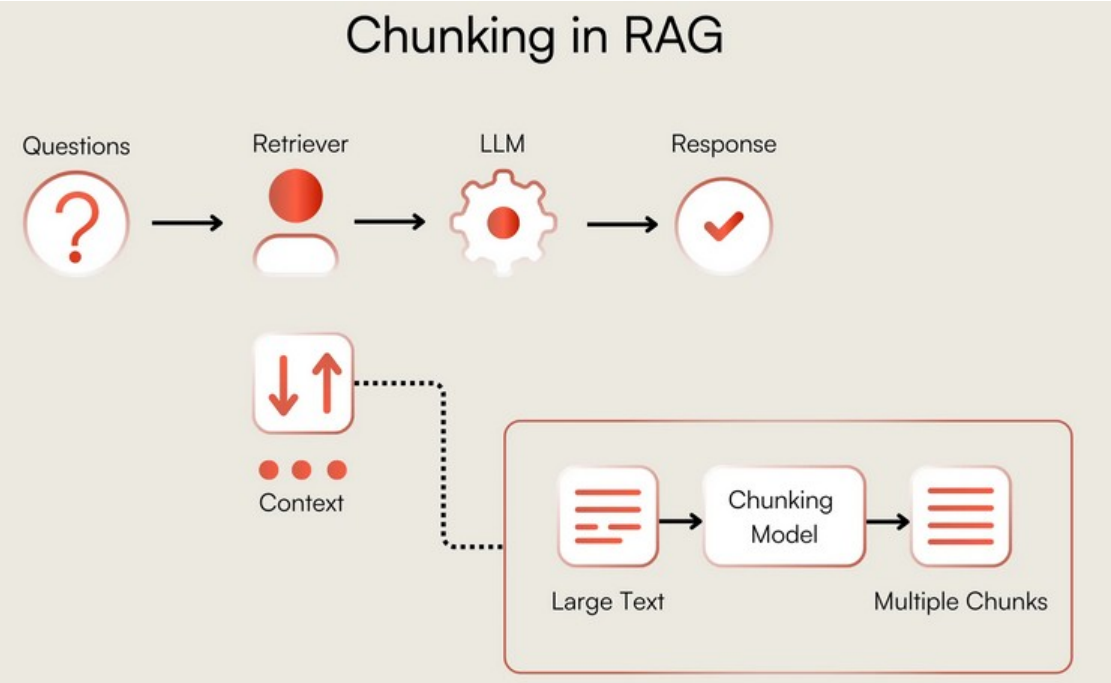


Image Credit: https://www.multimodal.dev/post/how-to-chunk-documents-for-rag

© 2026 by Innovation In Software Corporation

Chunking

- Code-specific
  - For repositories
    - Split by function
    - Split by class
    - Split by file
  - Rather than arbitrary token counts
  - Code chunking respects
    - Syntax trees
    - Logical program structure
  - AST-based chunking is often superior

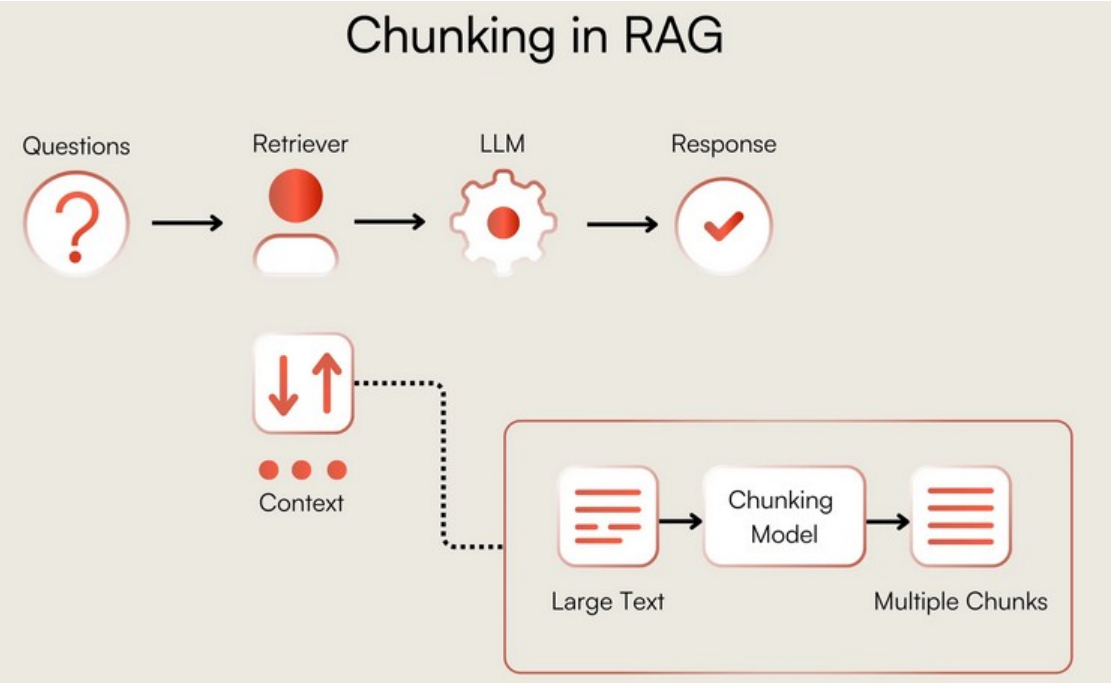


Image Credit: https://www.multimodal.dev/post/how-to-chunk-documents-for-rag

© 2026 by Innovation In Software Corporation

Chunking

- AST-based
  - Structured, parser-driven approach to code-specific chunking
    - Parses the code into an Abstract Syntax Tree
    - Extracts chunks aligned to syntactic nodes
    - Ensures syntactic completeness
    - It relies on a formal grammar and language parser
  - AST-based is stricter and more formal than code specific chunking

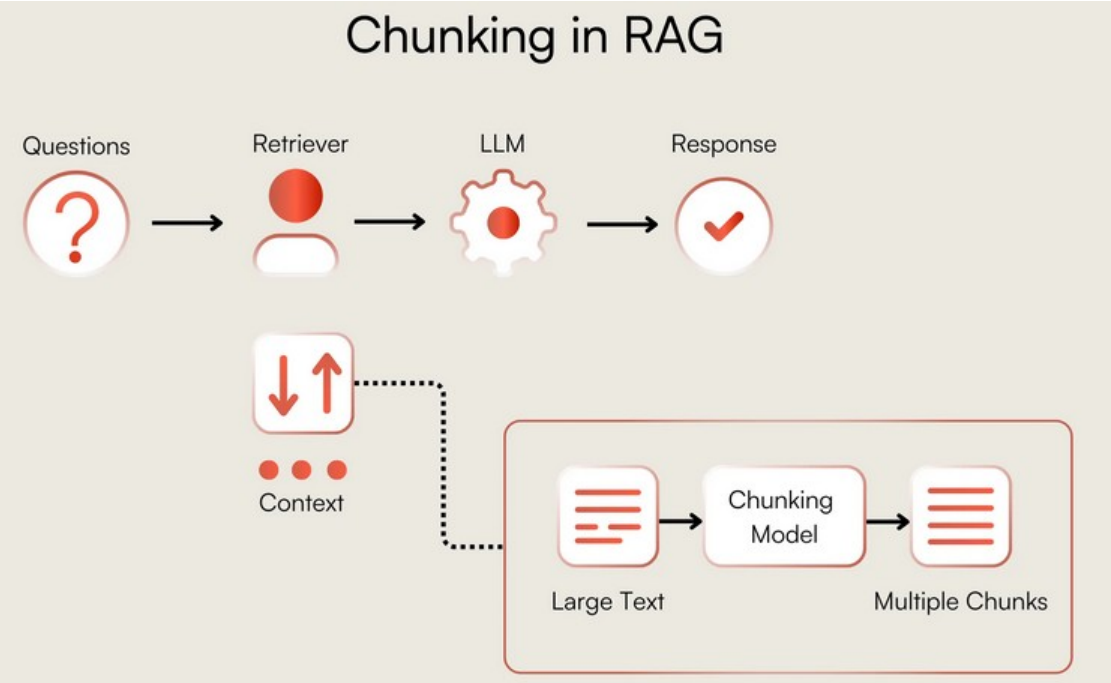


Image Credit: https://www.multimodal.dev/post/how-to-chunk-documents-for-rag

© 2026 by Innovation In Software Corporation

Embeddings

- Most production code RAG systems use a hybrid
  - Parse with AST
  - Extract
    - Function-level chunks
    - Class-level chunks
  - Add metadata
    - File path
    - Imports
    - Dependencies
  - Create higher-level file embeddings
- Used in serious code RAG systems

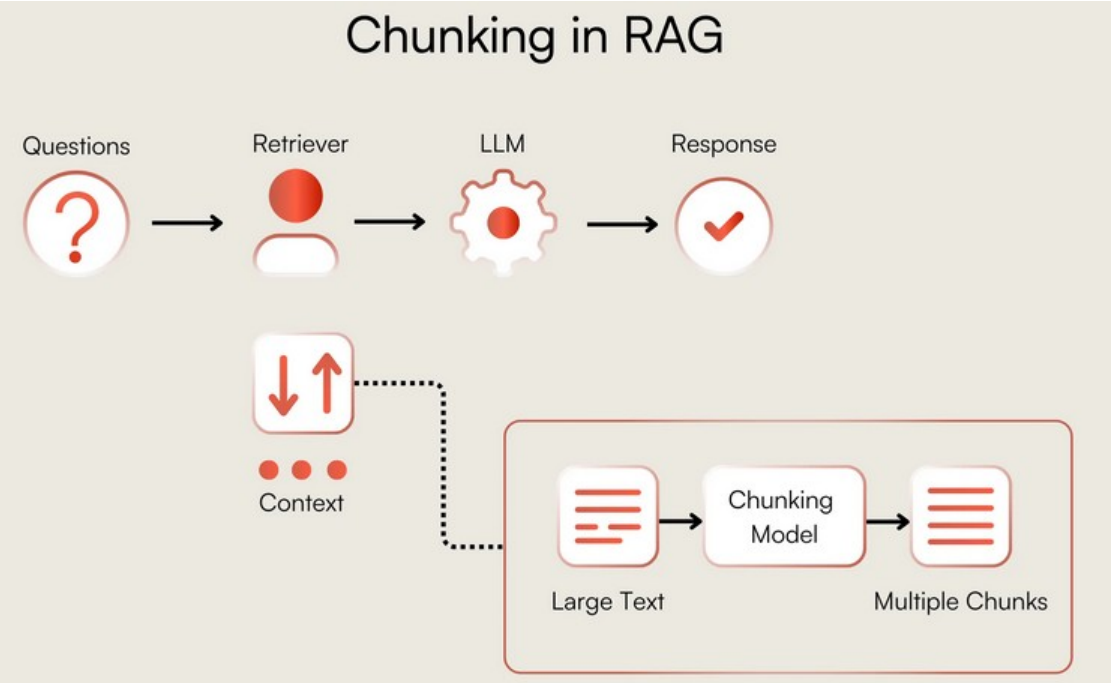


Image Credit: https://www.multimodal.dev/post/how-to-chunk-documents-for-rag

© 2026 by Innovation In Software Corporation

Embedding

- Embedding strategy
  - Representation design problem that directly affects
    - Retrieval recall
    - Retrieval precision
    - Latency
    - Storage
    - Downstream generation quality
  - The choice of embedding model defines the geometry of the knowledge space

Types of Embedding Models For RAG

Model type	Embeddings	Example model	Compute	Granularity/input
Sparse	0.3 [ ] 0.8	SPLADE	Low	Sentence, paragraph
Dense	0.3 0.1 0.8	Sentence transformers	Medium	Sentence, paragraph
Multivector	0.4 0.3 0.2 x tokens	COLBERT	High	Sentence, paragraph
Long context dense	0.3 0.1 0.8	text-embedding-3-small	Medium	Paragraphs, chapters
Variable dimension	0.3 0.1 0.8 Dimensions can change	text-embedding-3-small	Medium	Sentence, paragraph
Code (dense)	0.3 0.1 0.8	text-embedding-3-small	Medium	Functions, classes

Credits: Pratik Bhavsar, Galileo Labs

© 2026 by Innovation In Software Corporation

Embedding

- Each chunk
  - Is processed through a transformer encoder to turn it into a dense vector
  - The vector is a compressed semantic representation of the chunk
- The RAG system then
  - Stores the vector
  - Uses similarity metrics to retrieve nearby vectors
  - Embedding defines what “nearby” means

Types of Embedding Models For RAG

Model type	Embeddings	Example model	Compute	Granularity/input
Sparse	0.3 [ ] 0.8	SPLADE	Low	Sentence, paragraph
Dense	0.3 0.1 0.8	Sentence transformers	Medium	Sentence, paragraph
Multivector	0.4 0.3 0.2 x tokens	COLBERT	High	Sentence, paragraph
Long context dense	0.3 0.1 0.8	text-embedding-3-small	Medium	Paragraphs, chapters
Variable dimension	0.3 0.1 0.8 Dimensions can change	text-embedding-3-small	Medium	Sentence, paragraph
Code (dense)	0.3 0.1 0.8	text-embedding-3-small	Medium	Functions, classes

Credits: Pratik Bhavsar, Galileo Labs

© 2026 by Innovation In Software Corporation

Embedding

- Off-the-shelf embeddings
  - OpenAI embedding models
    - Proprietary models accessed via API
    - Trained at large scale
    - Optimized for general-purpose semantic similarity
  - SentenceTransformers
    - Open-source library built on top of BERT. RoBERTa etc
    - Created by the UKP Lab.
  - E5
    - E5 = “Embedding from Everything”
    - From MicroSoft

Types of Embedding Models For RAG

Model type	Embeddings	Example model	Compute	Granularity/input
Sparse	0.3 [ ] 0.8	SPLADE	Low	Sentence, paragraph
Dense	0.3 0.1 0.8	Sentence transformers	Medium	Sentence, paragraph
Multivector	0.4 0.3 0.2 x tokens	COLBERT	High	Sentence, paragraph
Long context dense	0.3 0.1 0.8	text-embedding-3-small	Medium	Paragraphs, chapters
Variable dimension	0.3 0.1 0.8 Dimensions can change	text-embedding-3-small	Medium	Sentence, paragraph
Code (dense)	0.3 0.1 0.8	text-embedding-3-small	Medium	Functions, classes

Credits: Pratik Bhavsar, Galileo Labs

© 2026 by Innovation In Software Corporation

Embedding

- Off-the-shelf embeddings
  - BGE
    - BAAI General Embedding
    - Beijing Academy of AI
  - Instructor models
    - Embed text conditioned on an instruction
- All are transformer encoder models
  - Trained with different objectives

Types of Embedding Models For RAG

Model type	Embeddings	Example model	Compute	Granularity/input
Sparse	0.3 [ ] 0.8	SPLADE	Low	Sentence, paragraph
Dense	0.3 0.1 0.8	Sentence transformers	Medium	Sentence, paragraph
Multivector	0.4 0.3 0.2 x tokens	COLBERT	High	Sentence, paragraph
Long context dense	0.3 0.1 0.8	text-embedding-3-small	Medium	Paragraphs, chapters
Variable dimension	0.3 0.1 0.8 Dimensions can change	text-embedding-3-small	Medium	Sentence, paragraph
Code (dense)	0.3 0.1 0.8	text-embedding-3-small	Medium	Functions, classes

Credits: Pratik Bhavsar, Galileo Labs

© 2026 by Innovation In Software Corporation

Embedding

Model Type	Hosted	Open Source	Customizable	Typical Use
OpenAI	Yes (API)	No	No	Production RAG
SentenceTransformers	No	Yes	Yes	Local / lightweight
E5	No	Yes	Yes	High-quality retrieval
BGE	No	Yes	Yes	State-of-the-art RAG
Instructor	No	Yes	Yes	Task-conditioned retrieval

© 2026 by Innovation In Software Corporation

Embedding

- Domain-specific
- Trained or fine-tuned on
  - Legal corpora
  - Biomedical corpora
  - Code repositories
  - Financial documents
- Advantages
  - Better domain recall
  - Improved semantic clustering
- Costs
  - Training complexity
  - Maintenance burden

Types of Embedding Models For RAG

Model type	Embeddings	Example model	Compute	Granularity/input
Sparse	0.3 [ ] 0.8	SPLADE	Low	Sentence, paragraph
Dense	0.3 0.1 0.8	Sentence transformers	Medium	Sentence, paragraph
Multivector	0.4 0.3 0.2 x tokens	COLBERT	High	Sentence, paragraph
Long context dense	0.3 0.1 0.8	text-embedding-3-small	Medium	Paragraphs, chapters
Variable dimension	0.3 0.1 0.8 Dimensions can change	text-embedding-3-small	Medium	Sentence, paragraph
Code (dense)	0.3 0.1 0.8	text-embedding-3-small	Medium	Functions, classes

Credits: Pratik Bhavsar, Galileo Labs

© 2026 by Innovation In Software Corporation

Embedding Similarity

- Defining what “similar” or “close” means for two vectors
- Embeddings live in high-dimensional vector space
  - Requires a metric to quantify nearness
- Cosine similarity - most common in RAG
  - Cosine similarity ignores magnitude and focuses on direction
  - In embeddings
    - Direction encodes semantic meaning
    - Magnitude often encodes less useful information

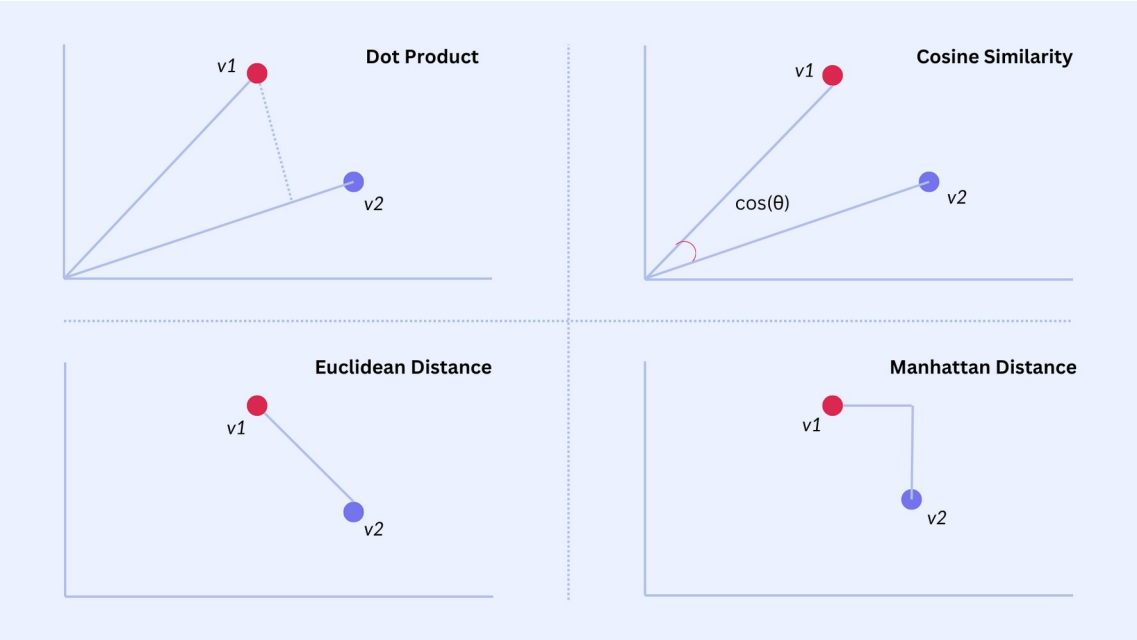


Image Credit: https://qdrant.tech/blog/what-is-vector-similarity/

© 2026 by Innovation In Software Corporation

Embedding Similarity

- Dot product (inner product)
  - Dot product increases when
  - Vectors align
  - Magnitudes are large
  - If vectors are normalized
    - Dot product approximates cosine similarity
- ANN libraries (e.g., FAISS) optimize dot product search efficiently

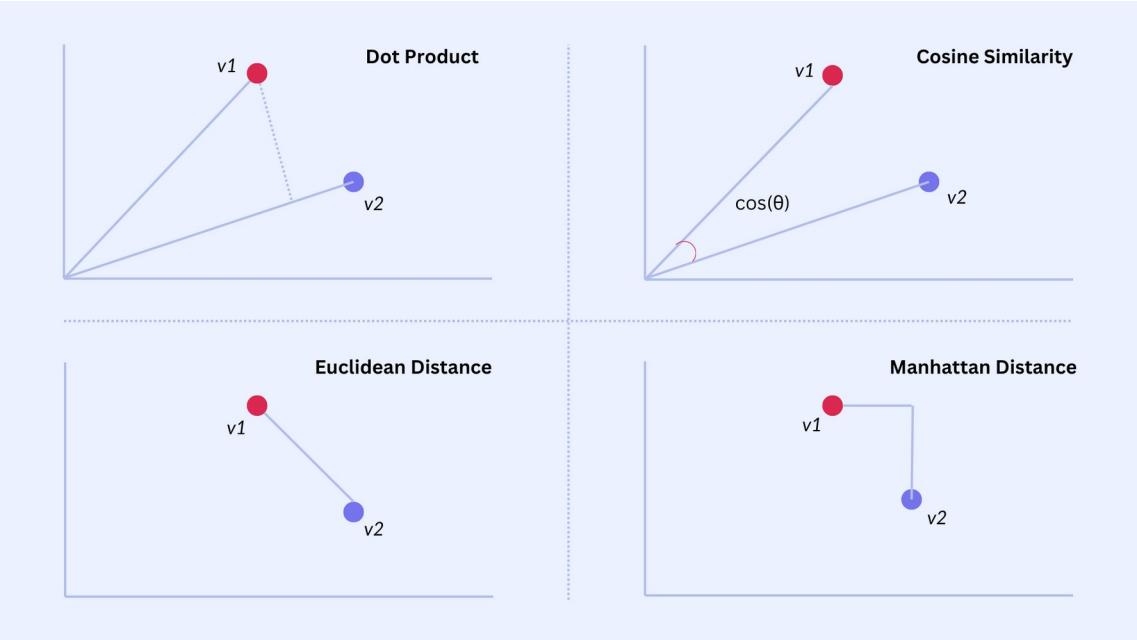


Image Credit: https://qdrant.tech/blog/what-is-vector-similarity/

© 2026 by Innovation In Software Corporation

Embedding Similarity

- Euclidean distance (L2 distance)
  - Straight-line distance between two points in n-dimensional space
  - The smaller distance, the more similar the vectors
- Drawbacks
  - High-dimensional vector spaces behave differently
  - Distances tend to concentrate
  - Angular similarity is often more meaningful than raw distance
  - Cosine often performs better empirically

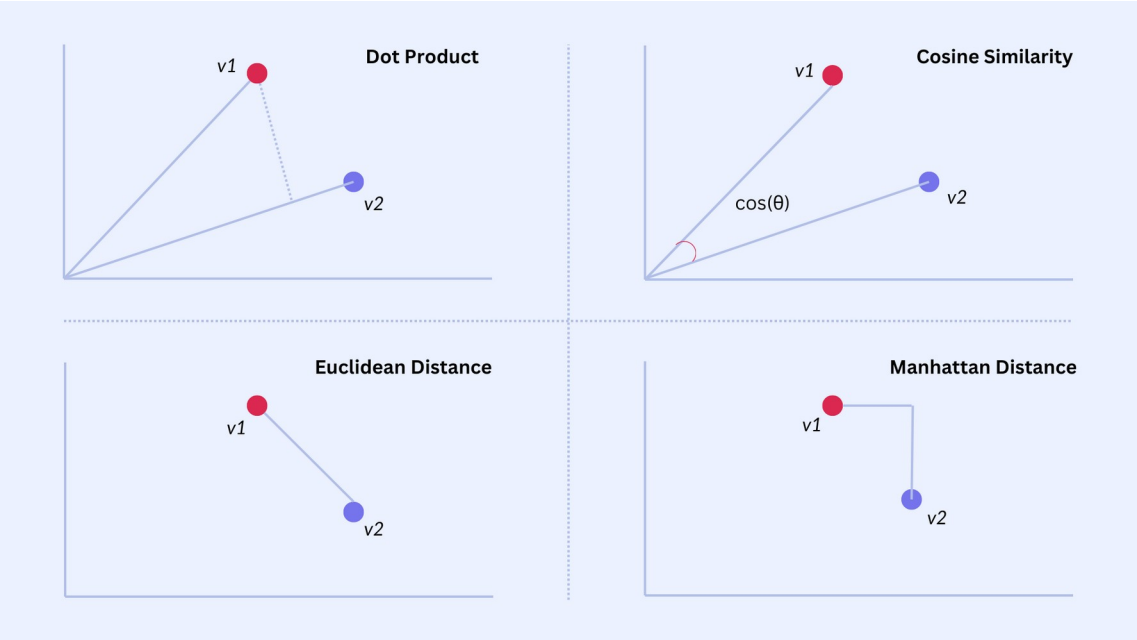


Image Credit: https://qdrant.tech/blog/what-is-vector-similarity/

© 2026 by Innovation In Software Corporation

Embedding Models

- There are several major categories of embedding models
  - Sparse embeddings
  - Dense embeddings
  - Contrastive-trained embeddings
  - Code embeddings
  - Hybrid embeddings
  - Late-interaction / multi-vector embeddings

Types of Embedding Models For RAG

Model type	Embeddings	Example model	Compute	Granularity/input
Sparse	0.3 | | 0.8	SPLADE	Low	Sentence, paragraph
Dense	0.3 | 0.1 | 0.8	Sentence transformers	Medium	Sentence, paragraph
Multivector	0.4 | 0.3 | 0.2 x tokens	COLBERT	High	Sentence, paragraph
Long context dense	0.3 | 0.1 | 0.8	text-embedding-3-small	Medium	Paragraphs, chapters
Variable dimension	0.3 | 0.1 | 0.8 Dimensions can change	text-embedding-3-small	Medium	Sentence, paragraph
Code (dense)	0.3 | 0.1 | 0.8	text-embedding-3-small	Medium	Functions, classes

Credits: Pratik Bhavsar, Galileo Labs

© 2026 by Innovation In Software Corporation

Embedding Models

- Sparse embeddings
  - Represent text as high-dimensional vectors where most values are zero
    - Vector dimension = vocabulary size
    - Most entries = 0
    - Only observed terms have non-zero weights
  - Advantages
    - Interpretable
    - Exact keyword matching
    - Strong for lexical search
  - Drawbacks
    - No semantic generalization
    - Cannot match synonyms well
  - Work well for
    - Legal search
    - Exact-match retrieval
    - Hybrid search systems

Types of Embedding Models For RAG

Model type	Embeddings	Example model	Compute	Granularity/input
Sparse	0.3 [ ] 0.8	SPLADE	Low	Sentence, paragraph
Dense	0.3 0.1 0.8	Sentence transformers	Medium	Sentence, paragraph
Multivector	0.4 0.3 0.2 x tokens	COLBERT	High	Sentence, paragraph
Long context dense	0.3 0.1 0.8	text-embedding-3-small	Medium	Paragraphs, chapters
Variable dimension	0.3 0.1 0.8 Dimensions can change	text-embedding-3-small	Medium	Sentence, paragraph
Code (dense)	0.3 0.1 0.8	text-embedding-3-small	Medium	Functions, classes

Credits: Pratik Bhavsar, Galileo Labs

© 2026 by Innovation In Software Corporation

Embedding Models

- Dense embeddings
  - Low-dimensional (384–3072 typically)
    - Continuous vectors
    - Almost all dimensions non-zeroThey encode semantic similarity
    - Will be close even without shared words
  - Advantages
    - Semantic generalization
    - Handles paraphrasing
    - Good for RAG
  - Disadvantages
    - Not interpretable
    - Requires ANN infrastructure
  - Most RAG systems use dense embeddings

Types of Embedding Models For RAG

Model type	Embeddings	Example model	Compute	Granularity/input
Sparse	0.3 [ ] 0.8	SPLADE	Low	Sentence, paragraph
Dense	0.3 0.1 0.8	Sentence transformers	Medium	Sentence, paragraph
Multivector	0.4 0.3 0.2 x tokens	COLBERT	High	Sentence, paragraph
Long context dense	0.3 0.1 0.8	text-embedding-3-small	Medium	Paragraphs, chapters
Variable dimension	0.3 0.1 0.8 Dimensions can change	text-embedding-3-small	Medium	Sentence, paragraph
Code (dense)	0.3 0.1 0.8	text-embedding-3-small	Medium	Functions, classes

Credits: Pratik Bhavsar, Galileo Labs

© 2026 by Innovation In Software Corporation

Embedding Models

- Code embeddings
  - Trained specifically for programming languages
- Contrastive embeddings
  - Part of how modern dense models are trained
  - The model tries to
    - Pull similar text together
    - Push unrelated text apart
  - Makes dense embeddings useful for retrieval

Types of Embedding Models For RAG

Model type	Embeddings	Example model	Compute	Granularity/input
Sparse	0.3 [ ] 0.8	SPLADE	Low	Sentence, paragraph
Dense	0.3 0.1 0.8	Sentence transformers	Medium	Sentence, paragraph
Multivector	0.4 0.3 0.2 x tokens	COLBERT	High	Sentence, paragraph
Long context dense	0.3 0.1 0.8	text-embedding-3-small	Medium	Paragraphs, chapters
Variable dimension	0.3 0.1 0.8 Dimensions can change	text-embedding-3-small	Medium	Sentence, paragraph
Code (dense)	0.3 0.1 0.8	text-embedding-3-small	Medium	Functions, classes

Credits: Pratik Bhavsar, Galileo Labs

© 2026 by Innovation In Software Corporation

Embedding Models

Type	Representation	Training Objective	Use Case
Sparse	High-dim, mostly zero	Frequency-based	Keyword search
Dense	Low-dim continuous	Contrastive	Semantic retrieval
Contrastive	Usually dense	Pull similar together	Modern RAG
Code	Dense, structured-aware	Code alignment tasks	Code search
Hybrid	Sparse + Dense	Combined scoring	Enterprise search
Multi-vector	Token-level dense	Late interaction	High-precision retrieval

© 2026 by Innovation In Software Corporation

Vector Databases

- The result of embedding will be a large collection of high dimension vectors
  - Scale could be billions of vectors
- Given a query vector, retrieval mean
  - Efficiently find the most similar vectors most to the query
- A vector databases support
  - High-dimensional similarity search
  - Fast nearest neighbor lookup
  - Scalable indexing
  - Metadata filtering

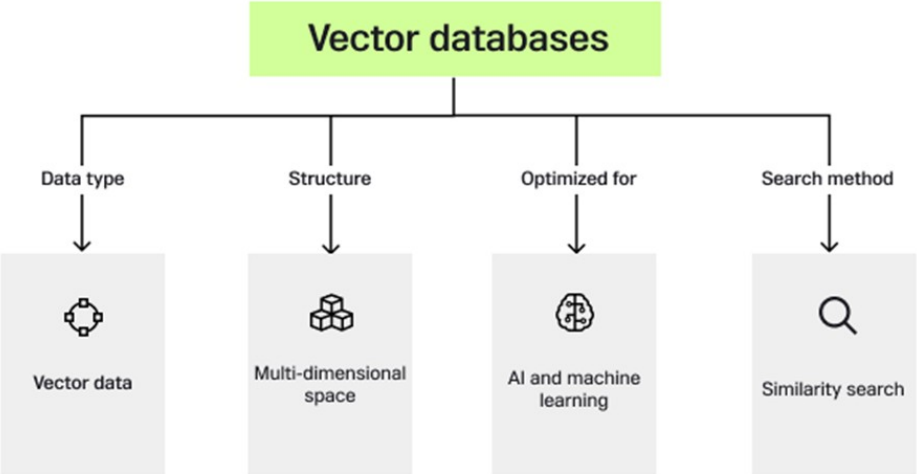


Image Credit: https://nexla.com/ai-infrastructure/vector-databases/

© 2026 by Innovation In Software Corporation

Vector Databases

- Efficient similarity search
  - Given a query ‘q’ we want to find the k most similar vectors to q in a set of N vectors
- Brute force search doesn’t work well
  - This would mean comparing q against all N vectors
    - Time complexity: O(N)
    - Not scalable at large N

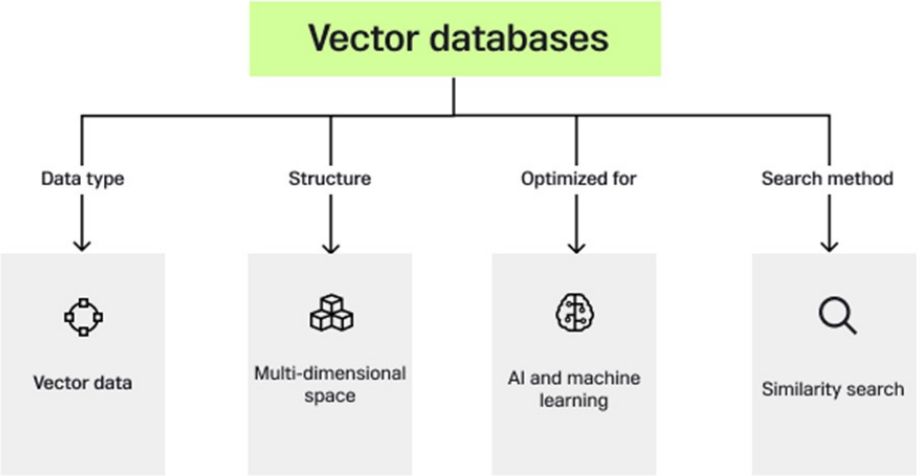


Image Credit: https://nexla.com/ai-infrastructure/vector-databases/

© 2026 by Innovation In Software Corporation

Vector Databases

- Embeddings can have 3000+ dimensions
  - Traditional databases are optimized for
    - Exact matches
    - Range queries
    - Structured filtering
  - They are not optimized for
    - Geometric nearest neighbor search in high-dimensional space.
- Vector databases are specifically engineered for this task

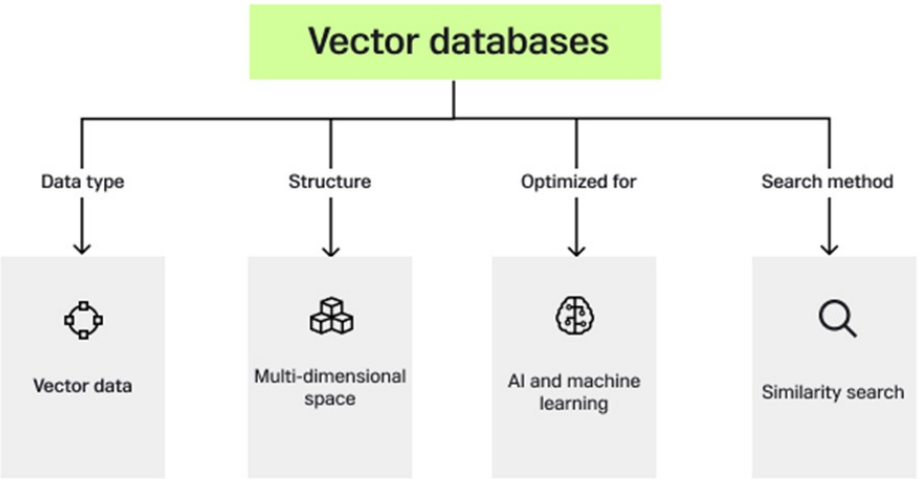


Image Credit: https://nexla.com/ai-infrastructure/vector-databases/

© 2026 by Innovation In Software Corporation

Approximate Nearest Neighbor

- Exact nearest neighbor search in high dimensions is
  - Computationally expensive and slow at scale
  - Becomes impractical in very large datasets
- Approximate Nearest Neighbor (ANN)
  - Returns vectors that are very close to the true nearest neighbors
    - Trades small accuracy loss for massive speed improvements
    - Instead of guaranteeing the mathematically exact nearest neighbor it finds neighbors with high probability of being near
  - Approximation error is usually negligible
  - Speed gains are dramatic

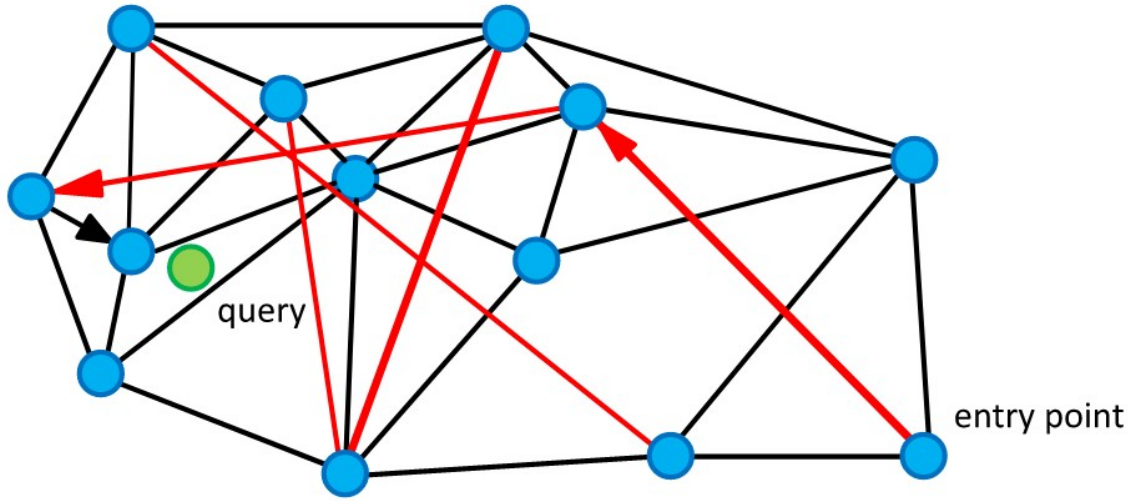


Image Credit: https://blog.vasnetsov.com/posts/categorical-hnsw/

© 2026 by Innovation In Software Corporation

Indexing Methods

- HNSW (Hierarchical Navigable Small World)
  - Graph-based index
    - Vectors are nodes in a graph
    - Edges connect nearby vectors
    - Multi-layer structure improves navigation
  - Search works by
    - Starting at top layer
    - Navigating down layers
    - Greedily moving toward closer neighbors
  - Strengths
    - Very high recall
    - Fast query time
    - Widely adopted
  - Common in
    - FAISS
    - Milvus
    - Weaviate
    - Pinecone

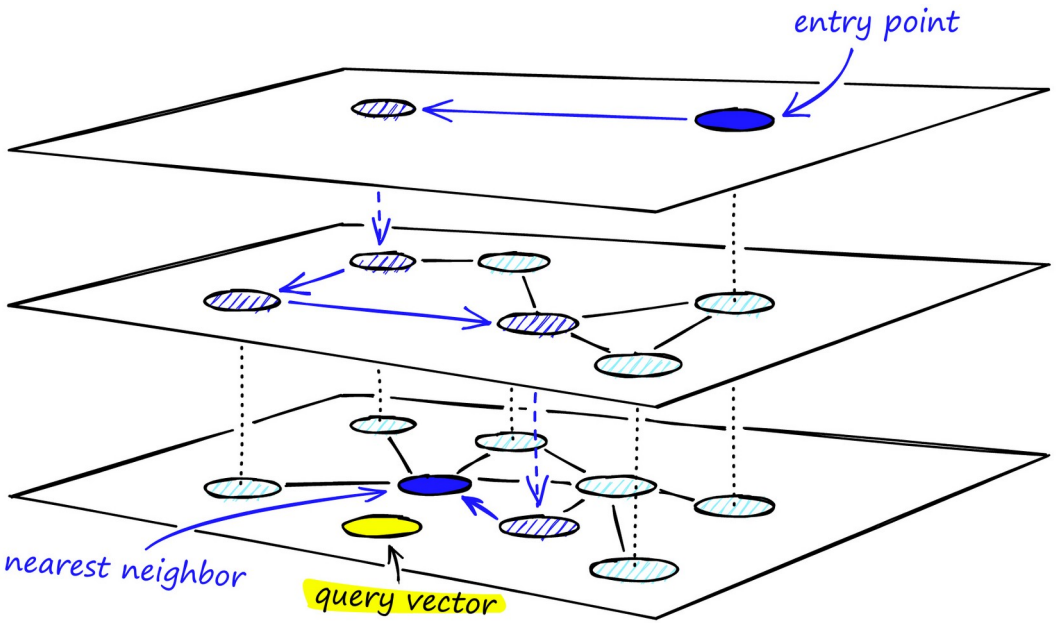


Image Credit: https://www.pinecone.io/learn/hnsw/

© 2026 by Innovation In Software Corporation

Indexing Methods

- IVF (Inverted File Index)
  - Cluster-based index
    - Run k-means clustering on dataset
    - Assign vectors to clusters (centroids)
  - At query time
    - Find nearest cluster(s)
    - Search only within those clusters
    - This reduces search space dramatically
  - Strengths
    - Good performance at large scale
    - Efficient memory usage
  - Trade-off
    - Lower recall if cluster routing misses relevant vectors

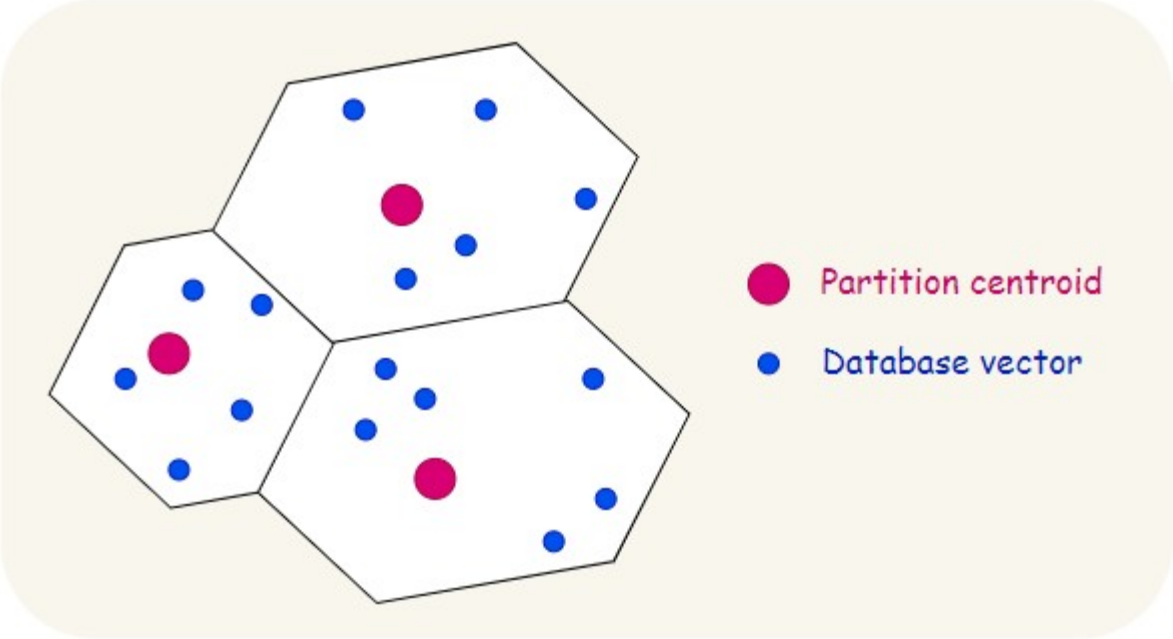
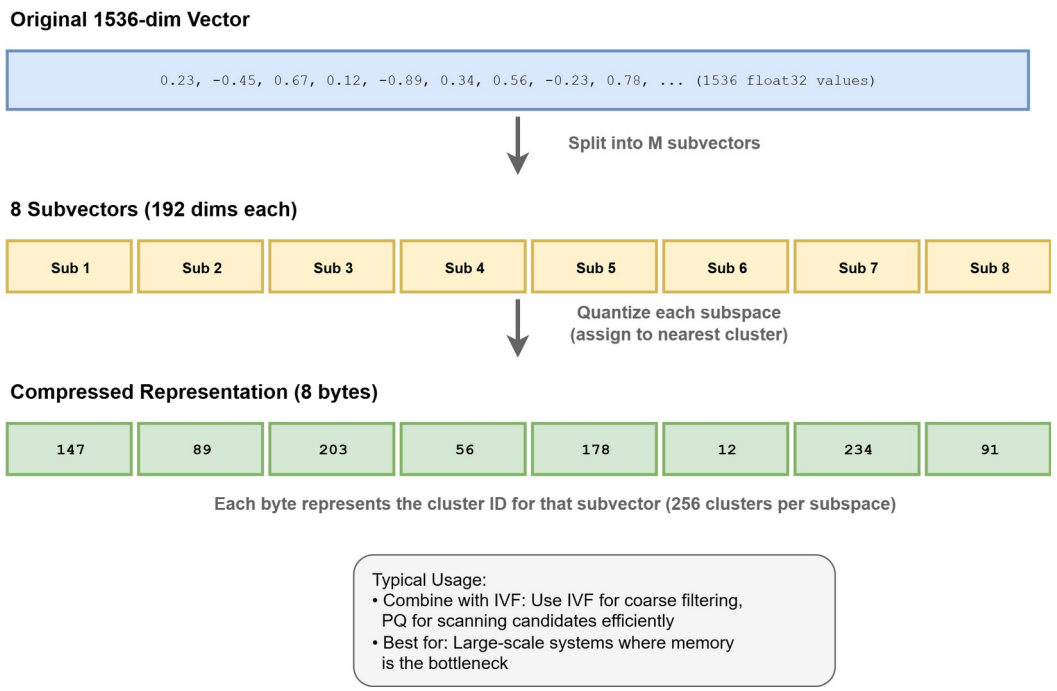


Image Credit: https://towardsdatascience.com/similarity-search-with-ivfpq-9c6348fd4db3

© 2026 by Innovation In Software Corporation

Indexing Methods

- Product quantization (PQ)
  - Not an index but used in IVF to improve efficiency
  - Compression-based technique
    - Compress vectors into smaller codes
    - Reduce memory footprint
    - Enable faster approximate distance computation
  - Used in billion-scale systems
  - Trade-off
    - Lower precision
    - Higher compression



© 2026 by Innovation In Software Corporation

Image Credit: https://booboone.com/the-complete-guide-to-vector-databases-for-machine-learning/

Approximate Nearest Neighbor

- Speed vs accuracy trade offs
- ANN systems can be tuned for
  - High accuracy
    - Slower search
    - Higher compute
    - More memory
  - High speed
    - Faster response
    - Slight recall drop
  - Vector databases allow adjusting
    - Number of search probes
    - Graph connectivity
    - Cluster count
    - Compression level

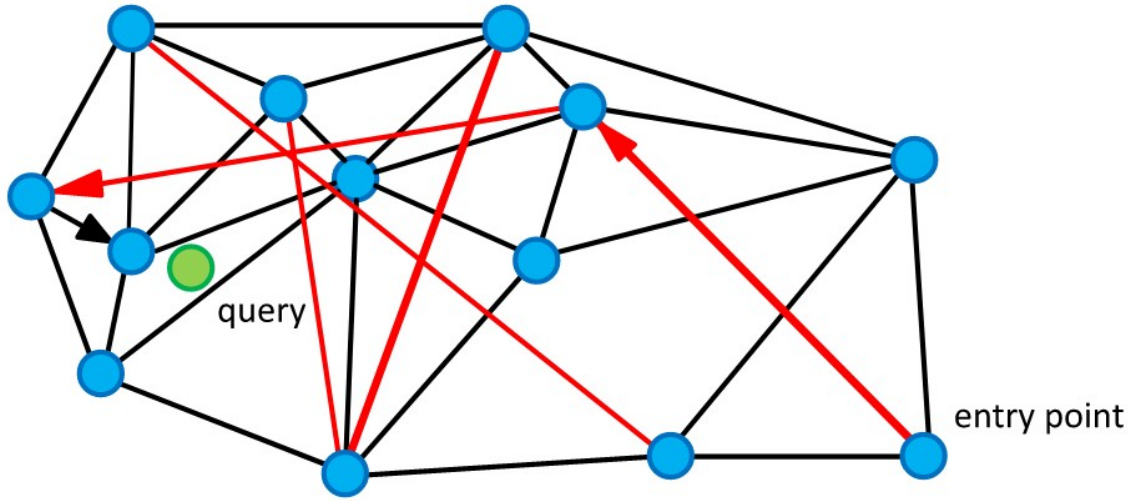


Image Credit: https://blog.vasnetsov.com/posts/categorical-hnsw/

© 2026 by Innovation In Software Corporation

Vector Database Products

- FAISS
  - Developed by Meta
    - High-performance similarity search library
    - Written in C++ with Python bindings
    - Supports GPU acceleration
  - Strengths:
    - Extremely fast
    - Highly configurable
    - Research-grade flexibility
  - Limitations:
    - Not a full database but a library
    - Often used inside larger systems

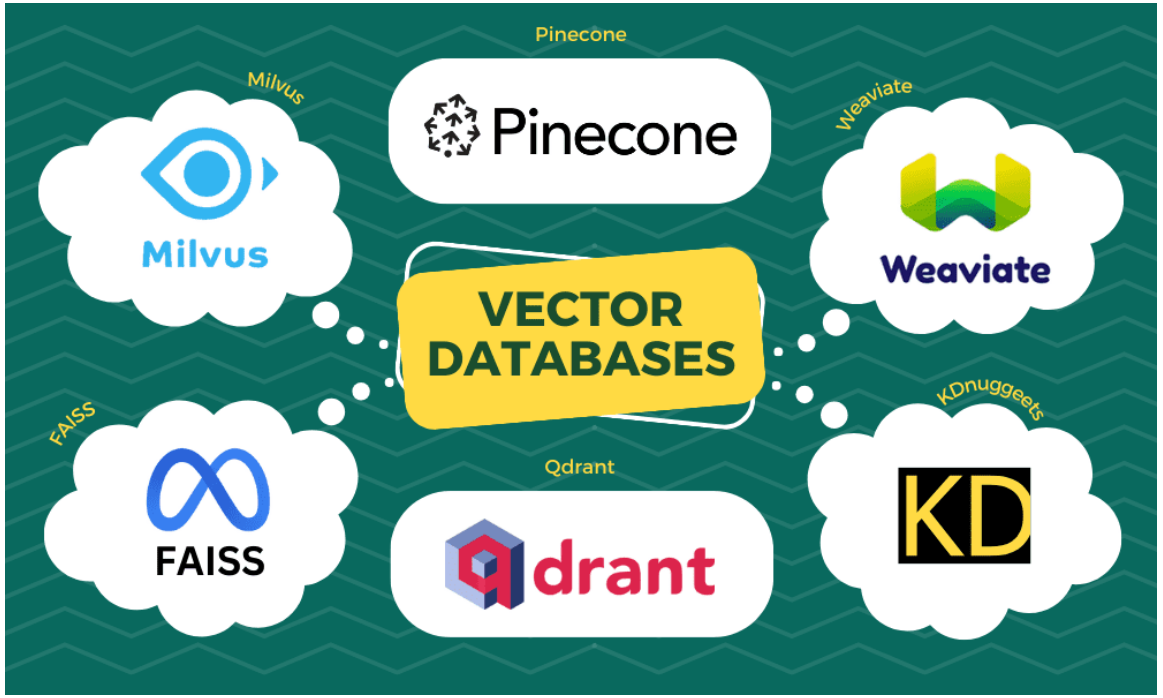


Image Credit: https://aiinteliigence.com/2023/11/17/the-5-best-vector-databases-you-must-try-in-2024/

© 2026 by Innovation In Software Corporation

Vector Database Products

- Pinecone
  - Managed vector database (cloud service)
    - Fully hosted infrastructure
    - Automatic scaling
    - Metadata filtering
    - High availability
  - Strengths
    - Production-ready
    - Easy integration
    - No infrastructure management
  - Trade-off
    - Cost
    - Less low-level control

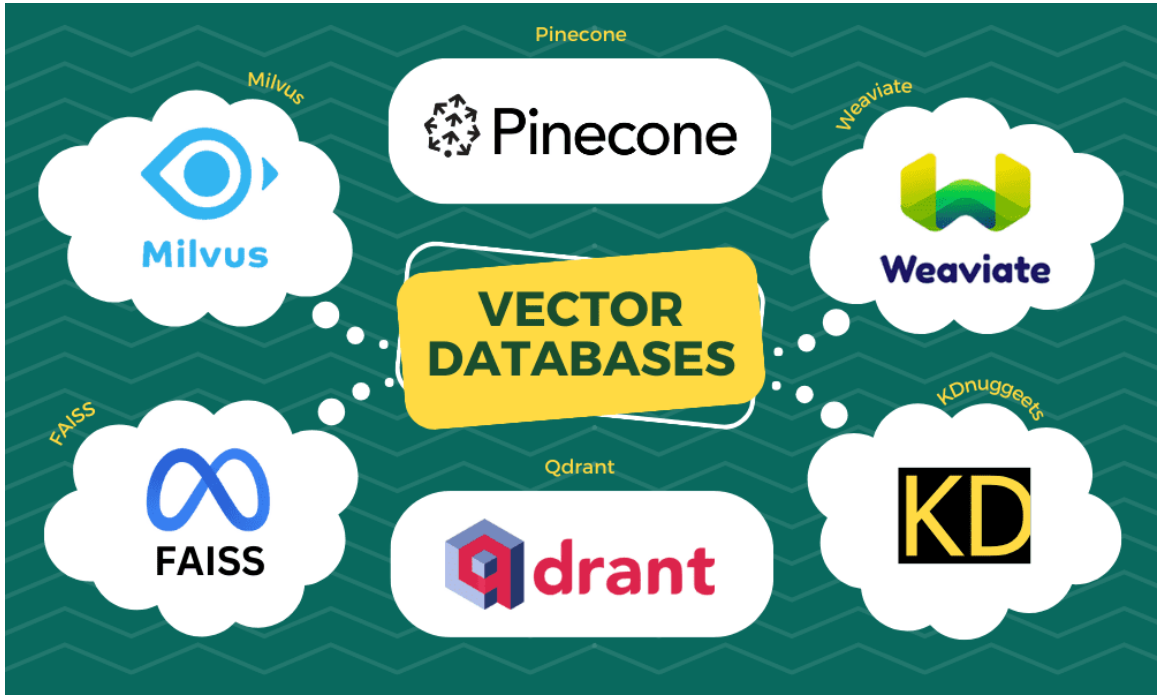


Image Credit: https://aiinteliigence.com/2023/11/17/the-5-best-vector-databases-you-must-try-in-2024/

© 2026 by Innovation In Software Corporation

Vector Database Products

- Weaviate
  - Open-source vector database
    - Native vector search
    - Hybrid search – sparse lexical search and dense semantic search
    - GraphQL API
    - Metadata filtering
  - Strengths
    - Flexible
    - Open-source deployment option
    - Strong hybrid search support

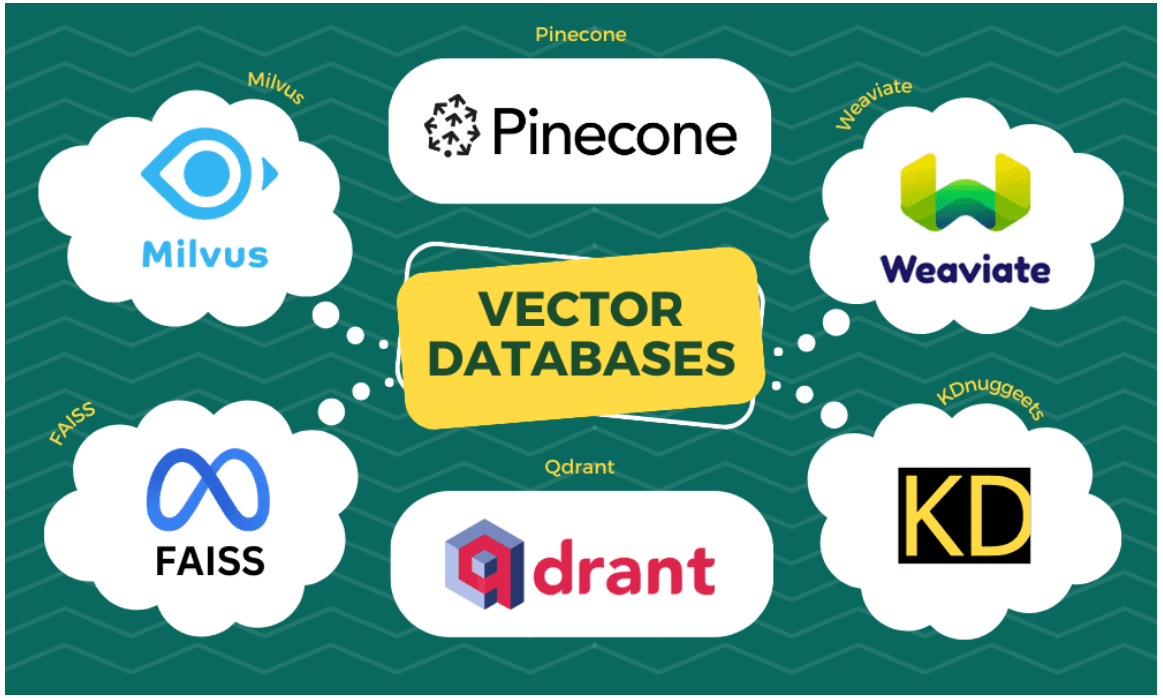


Image Credit: https://aiinteliigence.com/2023/11/17/the-5-best-vector-databases-you-must-try-in-2024/

© 2026 by Innovation In Software Corporation

Vector Database Products

- Milvus
  - Open-source vector database built for scale
    - Distributed architecture
    - Multiple ANN index types
    - Large-scale production deployments
  - Strengths
    - Scales to billions of vectors
    - Production-grade distributed system
    - Often used in enterprise AI systems.

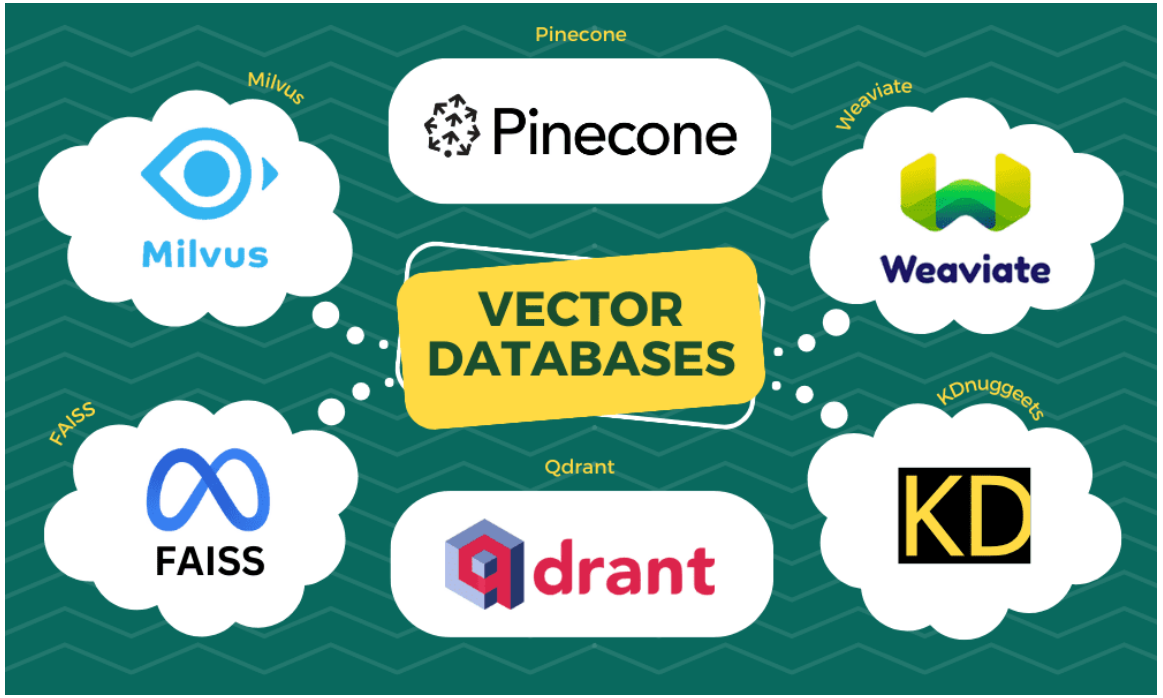


Image Credit: https://aiinteliigence.com/2023/11/17/the-5-best-vector-databases-you-must-try-in-2024/

© 2026 by Innovation In Software Corporation

Vector Database Products

- Chroma
  - Lightweight vector database designed for
    - RAG prototyping
    - Developer experimentation
  - Strengths
    - Easy setup
    - Python-first
    - Good for small-to-medium systems
  - Not optimized for billion-scale production

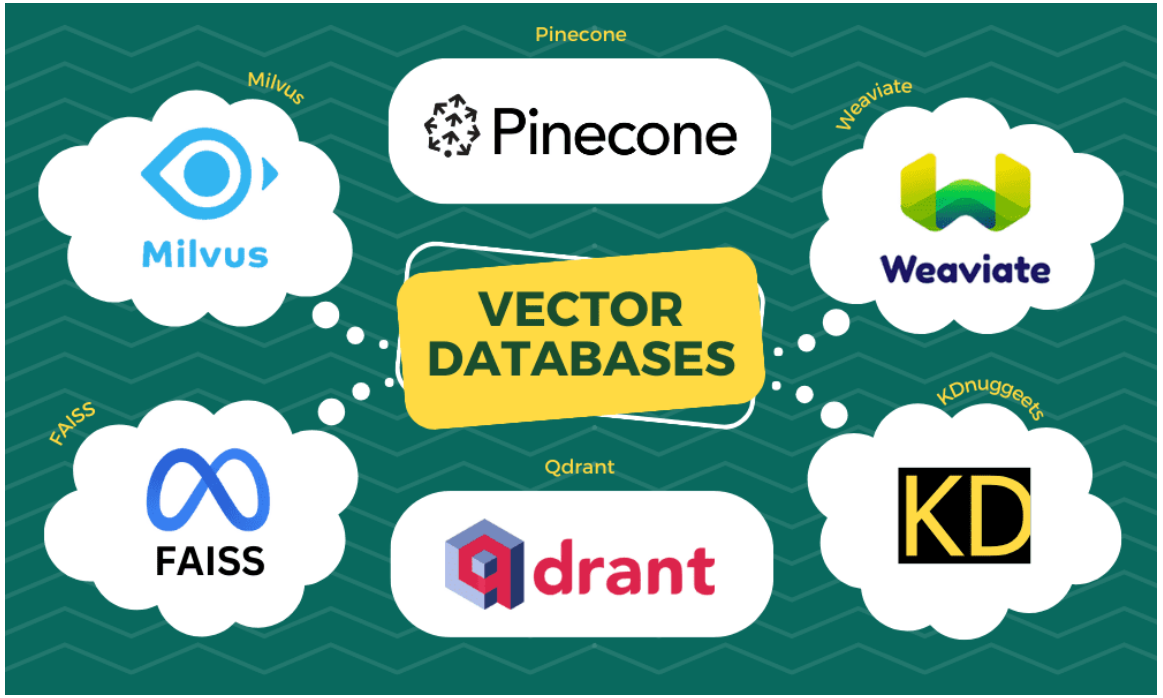


Image Credit: https://aiinteliigence.com/2023/11/17/the-5-best-vector-databases-you-must-try-in-2024/

© 2026 by Innovation In Software Corporation

Vector Database Products

- Considerations in choosing a vector db
  - Scale (thousands vs billions of vectors)
  - Latency requirements
  - Metadata filtering complexity
  - Hybrid search needs
  - Deployment model
    - managed vs self-hosted
  - Cost constraints

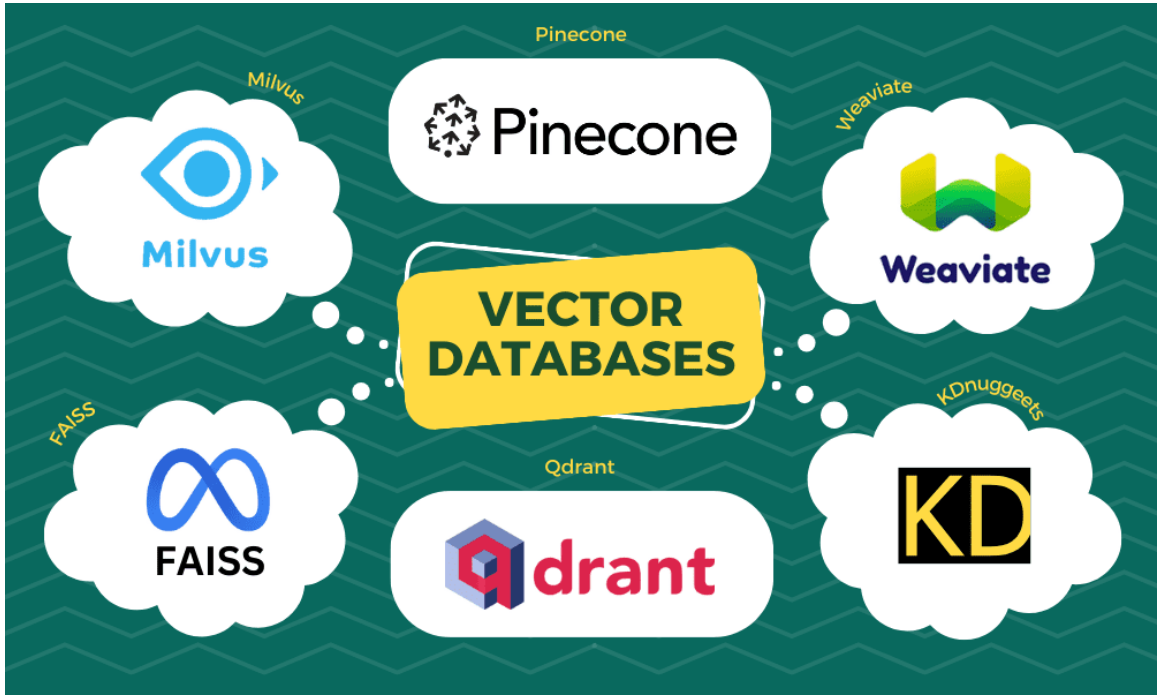


Image Credit: https://aiinteliigence.com/2023/11/17/the-5-best-vector-databases-you-must-try-in-2024/

© 2026 by Innovation In Software Corporation

Retrieval Strategies

- Top-k retrieval
  - Simplest and most common retrieval method.
  - Given
    - A query embedding
    - A vector database
    - A similarity measure (for example, cosine)
  - Returns
    - The k most similar vectors
    - Where k is the number of results to be returned

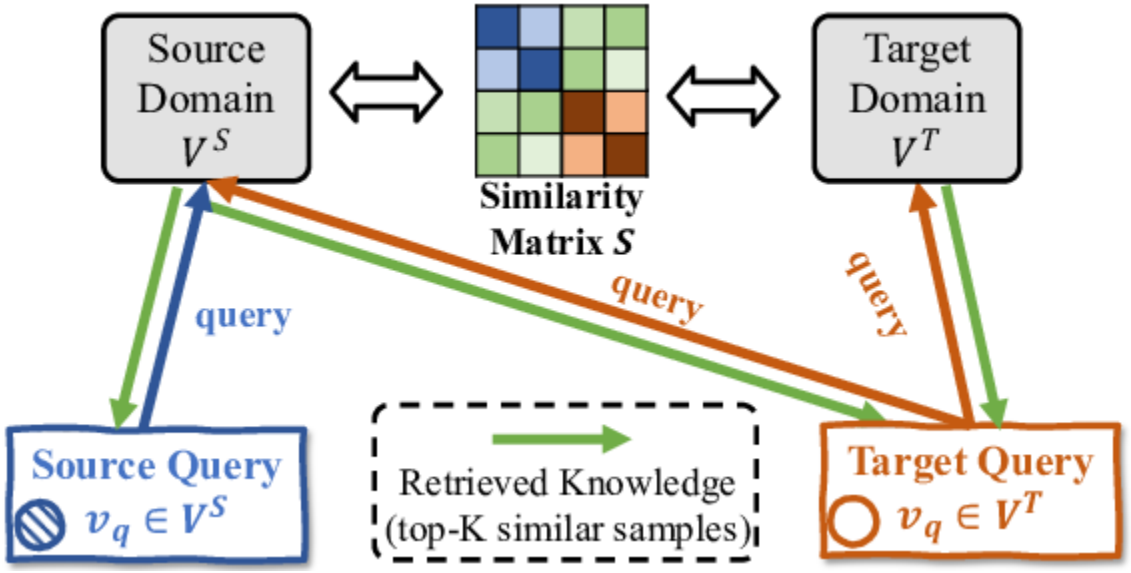
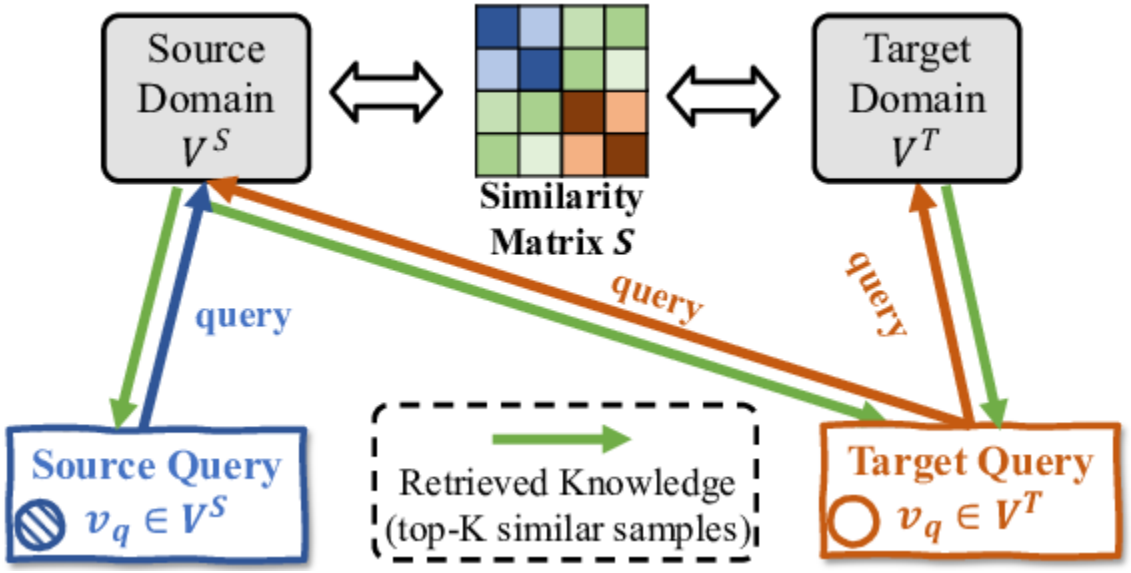


Image Credit: https://www.researchgate.net/figure/Knowledge-retrieval-top-K-beneficial-samples_fig4_373246496

© 2026 by Innovation In Software Corporation

Retrieval Strategies

- Choosing k
  - Small k
    - Faster
    - Lower token cost
    - Risk missing relevant evidence
  - Large k
    - Higher recall
    - More noise
    - Increased latency and LLM cost
  - Typical production values
    - k = 3–10 for small corpora
    - k = 10–50 for large corpora with re-ranking



© 2026 by Innovation In Software Corporation

Image Credit: https://www.researchgate.net/figure/Knowledge-retrieval-top-K-beneficial-samples_fig4_373246496

Retrieval Strategies

- Limitations
  - Assumes embedding similarity = relevance
  - Sensitive to embedding quality
  - May retrieve near-duplicates

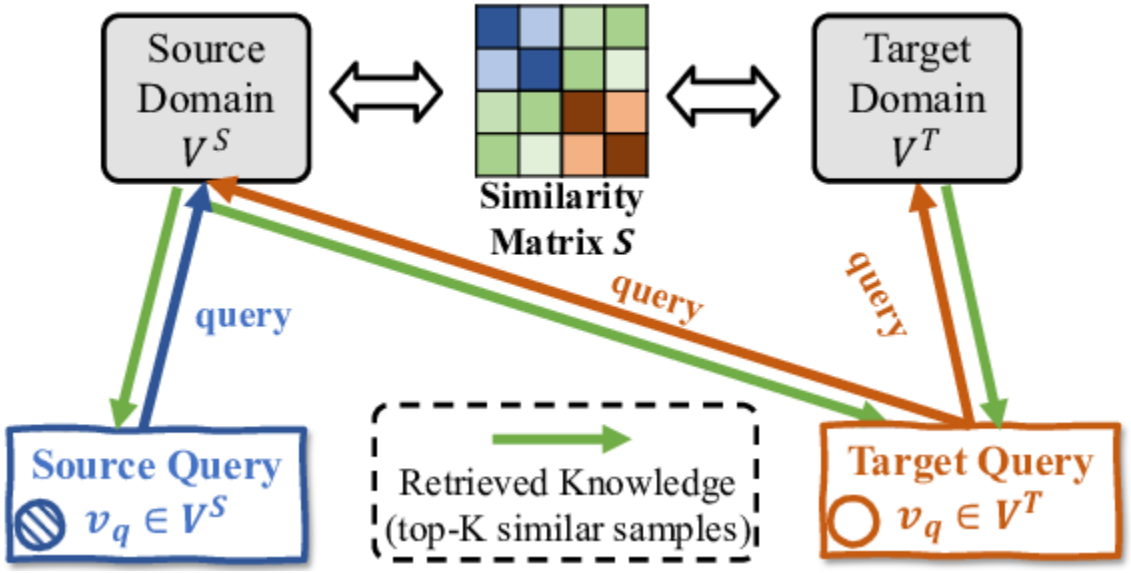
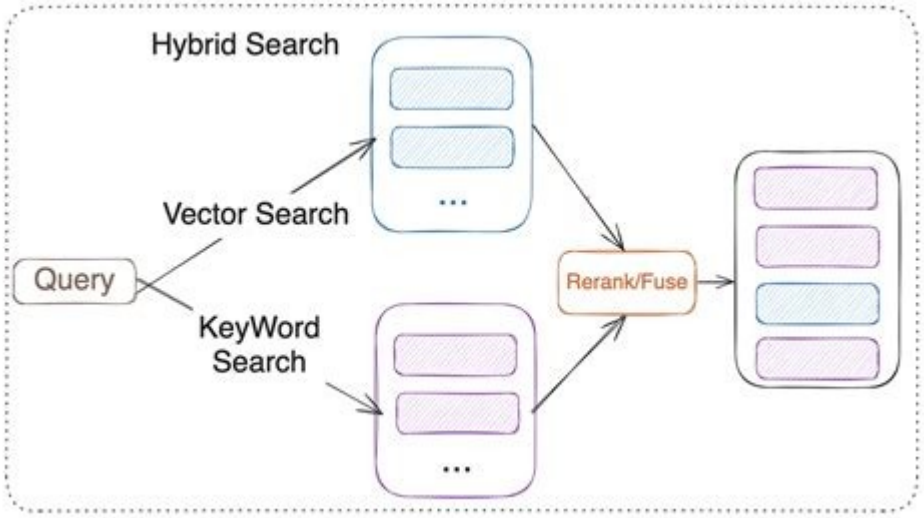


Image Credit: https://www.researchgate.net/figure/Knowledge-retrieval-top-K-beneficial-samples_fig4_373246496

© 2026 by Innovation In Software Corporation

Retrieval Strategies

- Hybrid search (vector + keyword)
  - Combines
    - Dense vector similarity (semantic search)
    - Sparse keyword search (e.g., BM25)
  - Score=α·DenseSim+β·KeywordScore
  - Dense embeddings
    - Capture paraphrases
    - Capture semantic meaning
    - May miss exact terms
    - May ignore critical identifiers
  - Keyword search
    - Excellent at exact match
    - Weak at semantic generalization
  - Hybrid search balances both



© 2026 by Innovation In Software Corporation

Image Credit: https://docs.vectorchord.ai/vectorchord/use-case/hybrid-search.html

Retrieval Strategies

- Hybrid search (vector + keyword)
  - When hybrid is critical
    - Legal documents
    - Regulatory environments
    - Technical specs
    - Enterprise document search
    - Anywhere exact terminology matters
  - Benefits
    - Improved recall
    - Better precision
    - Robustness to query phrasing
  - Hybrid search is becoming the default in enterprise RAG

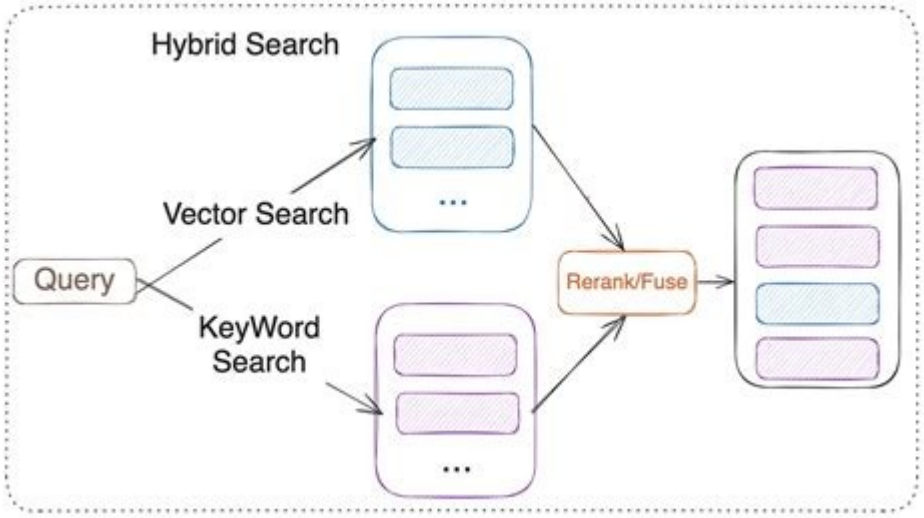


Image Credit: https://docs.vectorchord.ai/vectorchord/use-case/hybrid-search.html

© 2026 by Innovation In Software Corporation

Retrieval Strategies

- Re-ranking models
  - Two-stage retrieval pipeline
  - Stage 1
    - Fast ANN search
    - Retrieve top 50–100 candidates
  - Stage 2
    - Apply more accurate model
    - Re-rank candidates
    - Select top N (e.g., 5)
- Why use re-ranking?
  - ANN prioritizes speed
  - Re-ranking prioritizes precision
  - It filters out
    - Semantically similar but irrelevant chunks
    - Near-duplicates
    - Partially relevant matches

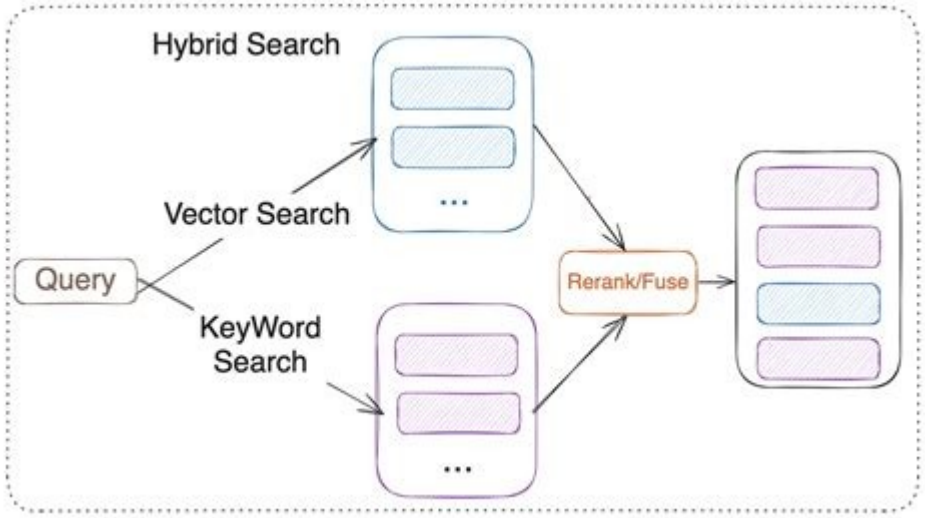


Image Credit: https://docs.vectorchord.ai/vectorchord/use-case/hybrid-search.html

© 2026 by Innovation In Software Corporation

Retrieval Strategies

- Cross-encoder re-ranking
  - In RAG systems, retrieval steps
    - Encode query into a vector
    - Compare to document vectors
    - Retrieve top-k using cosine similarity
  - This uses a bi-encoder architecture
    - Query encoded independently
    - Documents encoded independently
    - Similarity computed in vector space
  - This is efficient but limited
    - The model never directly compares query and document tokens
    - Interaction happens only through vector similarity.

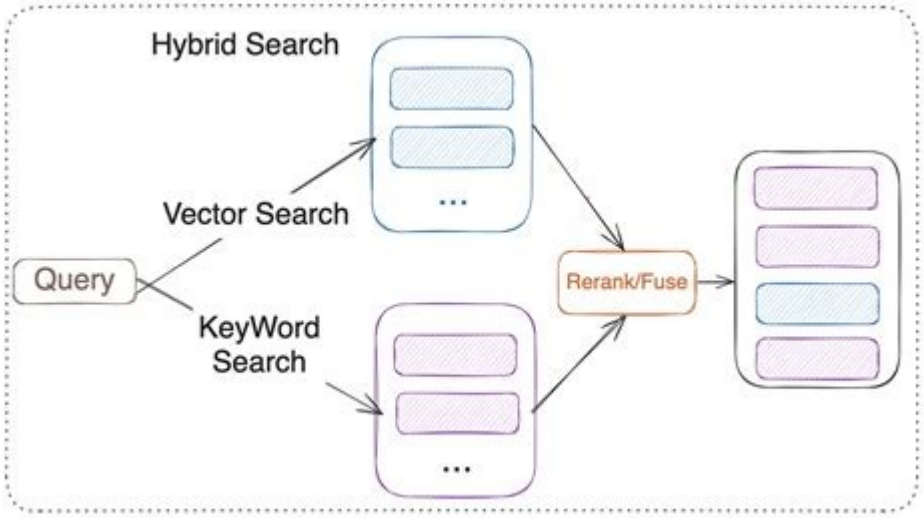


Image Credit: https://docs.vectorchord.ai/vectorchord/use-case/hybrid-search.html

© 2026 by Innovation In Software Corporation

Retrieval Strategies

- Cross-encoder re-ranking
  - Cross-Encoder
    - Evaluates the document in the context of the query, not in isolation
    - Compute relevance score - how relevant this document is to this query
  - More accurate
    - Captures fine-grained token interactions
    - Understands term dependencies
    - Handles negation and context better
  - Drawbacks
    - Computationally expensive
    - Cannot precompute embeddings
    - Requires running model per candidate result
    - Used only on small candidate sets.

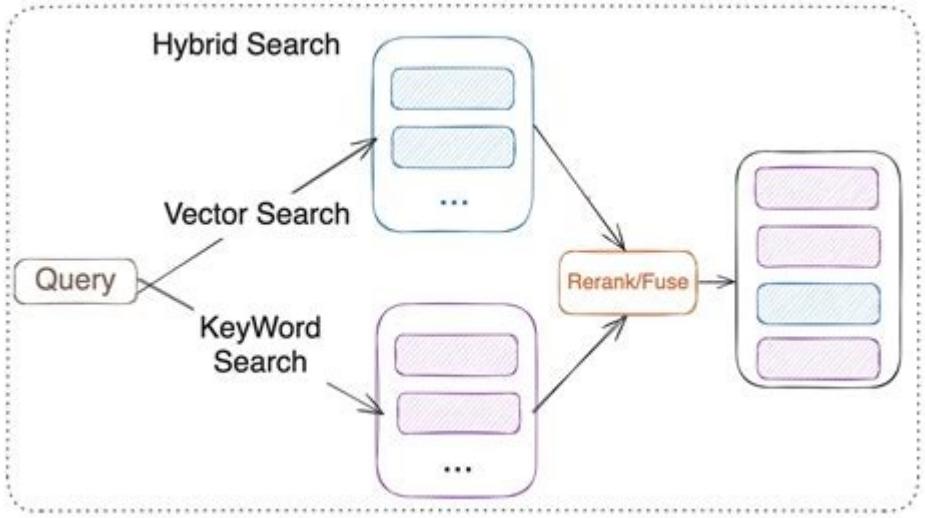


Image Credit: https://docs.vectorchord.ai/vectorchord/use-case/hybrid-search.html

© 2026 by Innovation In Software Corporation

Retrieval Strategies

- Filtering by metadata
  - Restrict retrieval to documents that satisfy structured constraints
    - Only retrieve HR policies
    - Only documents from 2024
    - Only documents user is authorized to access
  - Embedding similarity alone
    - Does not enforce governance
    - Does not enforce security
    - Does not reflect business logic
  - Metadata filtering ensures
    - Compliance
    - Access control
    - Contextual correctness
    - Implementation
  - Filtering can happen:
    - Before ANN search (pre-filtering)
    - During search (filtered index)
    - After retrieval (post-filtering)
  - Enterprise systems rely heavily on metadata filtering

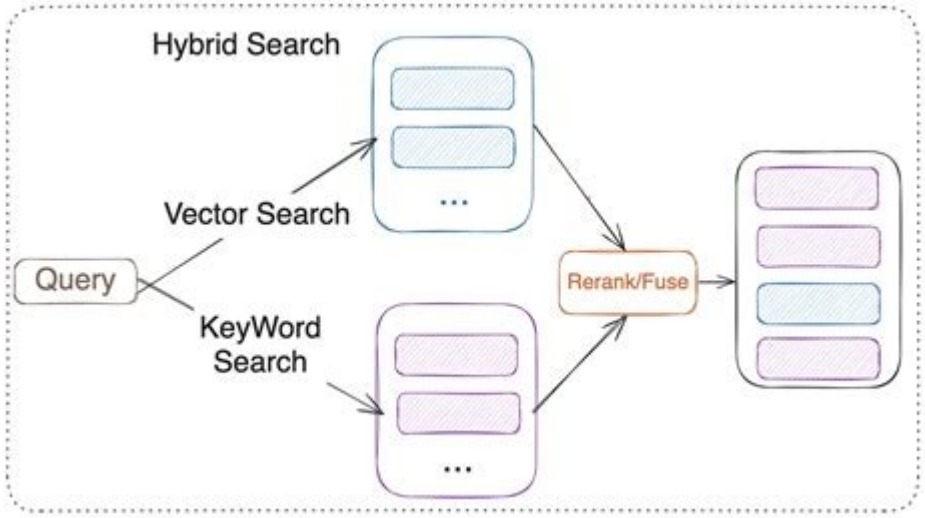


Image Credit: https://docs.vectorchord.ai/vectorchord/use-case/hybrid-search.html

© 2026 by Innovation In Software Corporation

Prompt Construction

- Retrieval alone does not guarantee good answers
  - The way retrieved context is inserted into the prompt significantly affects
    - Accuracy
    - Hallucination rate
    - Faithfulness
    - Token usage
    - Citation quality
  - Prompt construction is the bridge between retrieval and generation

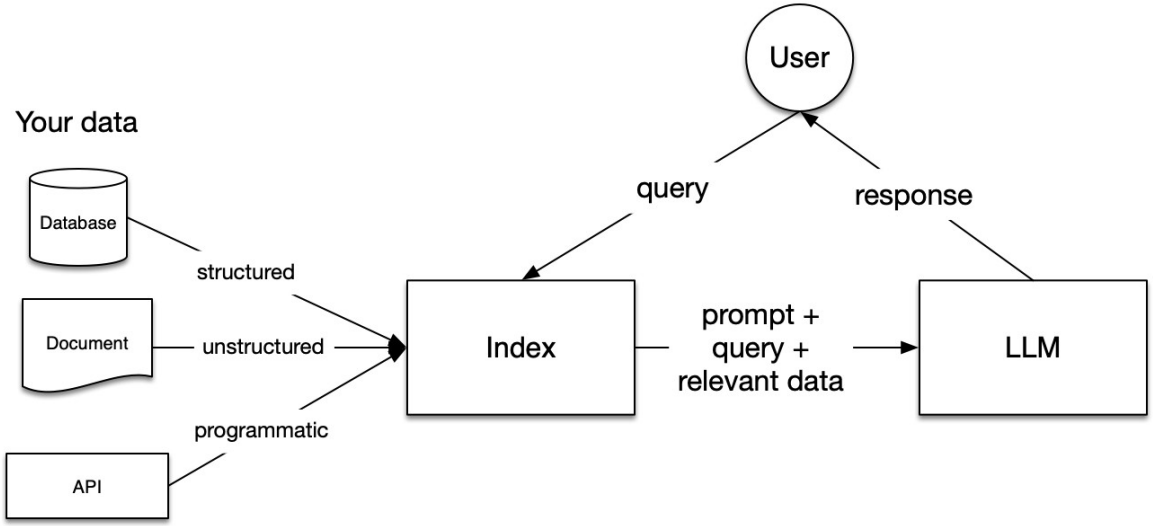


Image Credit: https://medium.com/@tselvaraj/the-role-of-prompt-engineering-in-rag-systems-f8aaf6f80327

© 2026 by Innovation In Software Corporation

Prompt Construction

- Context injection is
  - Placing retrieved documents into the LLM prompt so the model can condition on them during generation
  - Standard RAG uses in-context learning
  - Retrieved text is added to the prompt
  - The model uses it without weight updates
- The model does not “know” which content is more important
  - Prompt structure helps guide attention

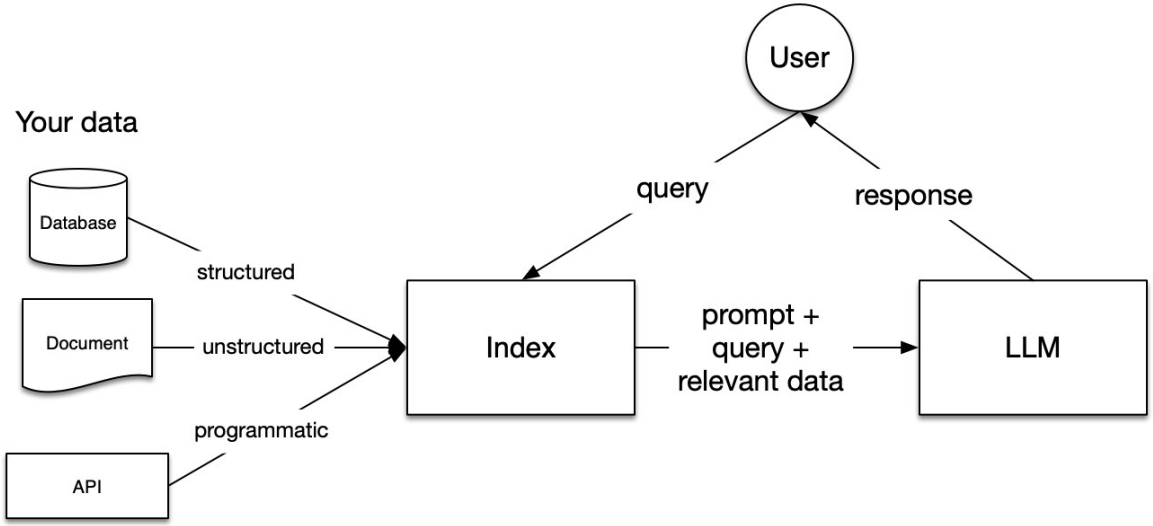


Image Credit: https://medium.com/@tselvaraj/the-role-of-prompt-engineering-in-rag-systems-f8aaf6f80327

© 2026 by Innovation In Software Corporation

Prompt Construction

- Citation formatting in prompts
  - Citations improve
    - Trust
    - Auditability
    - Governance
    - Debugging
  - Examples:
    - *Employees are entitled to 20 days of leave per year [Doc 1]*
    - *According to the 2024 HR Policy Manual*
    - *Sources: [1] HR Policy Manual (2024) [2] Employee Handbook (2023)*

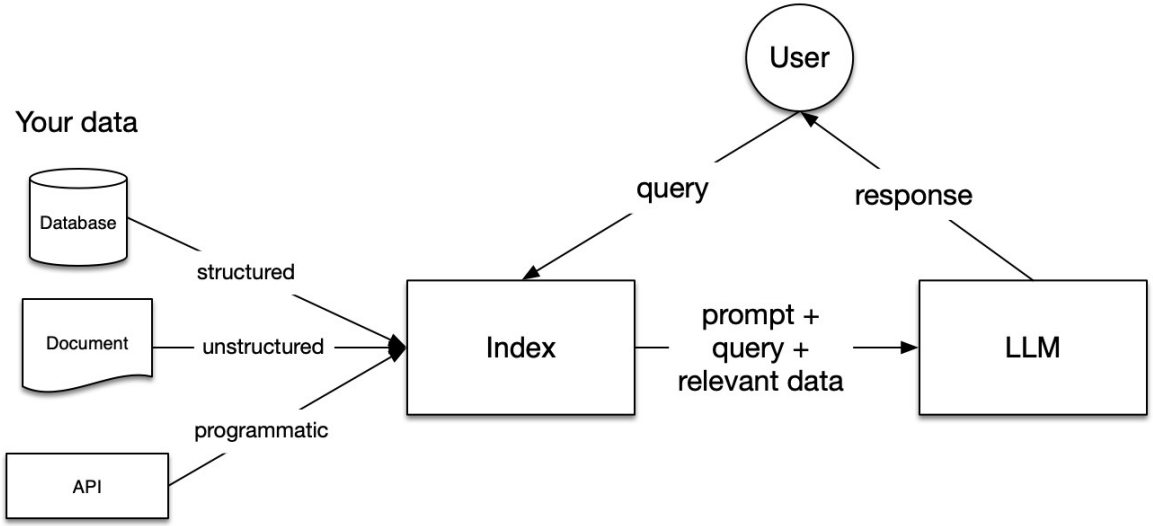


Image Credit: https://medium.com/@tselvaraj/the-role-of-prompt-engineering-in-rag-systems-f8aaf6f80327

© 2026 by Innovation In Software Corporation

Prompt Construction

- Citation formatting in prompts
  - Encourage grounded answers
  - Reduce hallucination tendency
  - Enable human verification
  - Support compliance requirements
  - Some systems require the model to quote directly from context

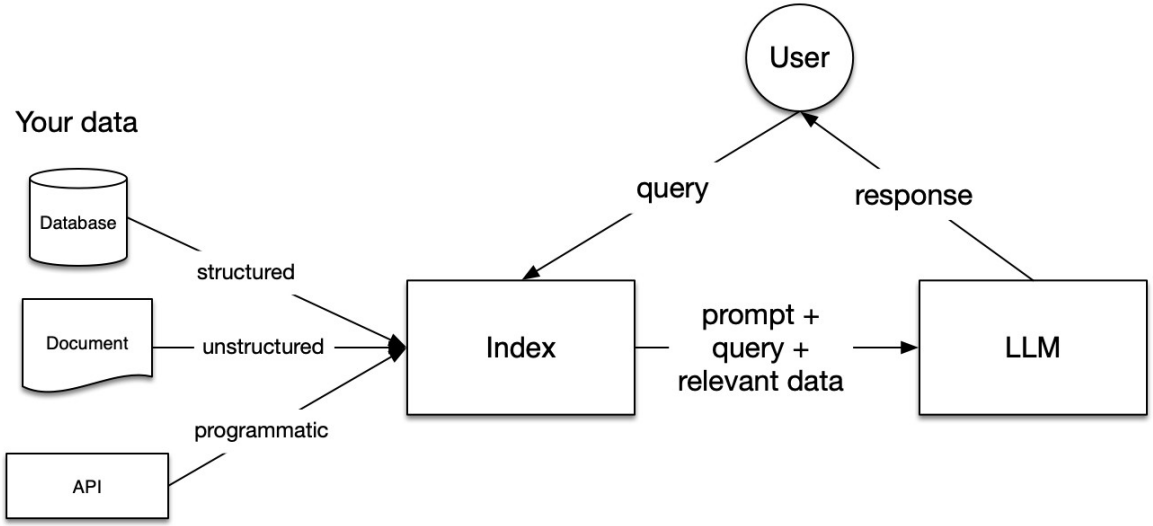


Image Credit: https://medium.com/@tselvaraj/the-role-of-prompt-engineering-in-rag-systems-f8aaf6f80327

© 2026 by Innovation In Software Corporation

Prompt Construction

- Limiting hallucinations
  - Even with retrieved context, models may
    - Add unsupported facts
    - Fill missing gaps
    - Extrapolate beyond evidence
- Prompt design can reduce this
  - Explicit grounding instructions
    - *Use only the provided context. If the answer is not in the context, say "I do not know." Do not add information not supported by the documents.*
  - This reduces hallucinations but does not eliminate them

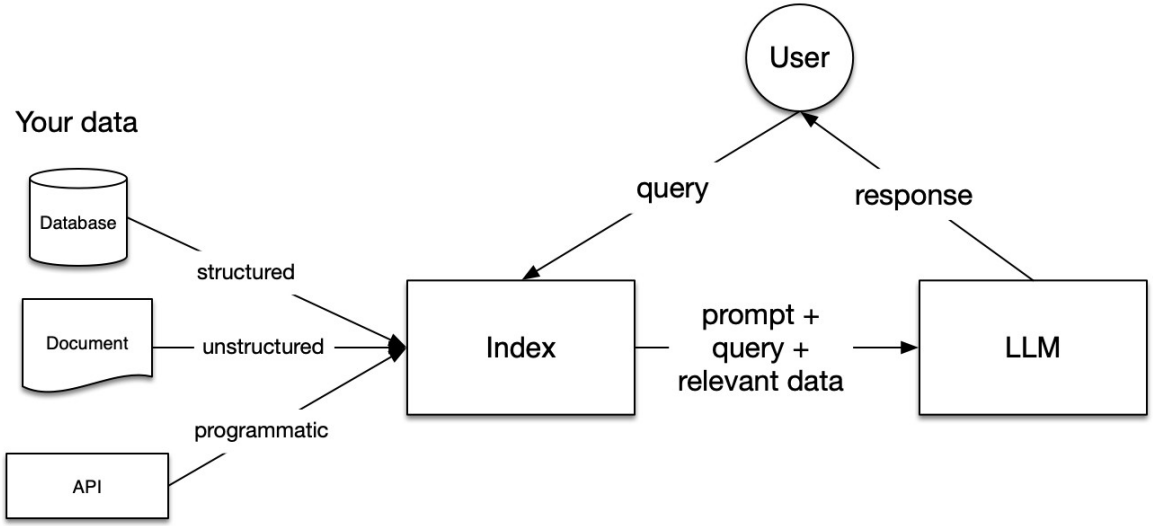


Image Credit: https://medium.com/@tselvaraj/the-role-of-prompt-engineering-in-rag-systems-f8aaf6f80327

© 2026 by Innovation In Software Corporation

Prompt Construction

- Evidence-first answering
  - Ask the model to
    - Identify relevant excerpts
    - Then answer
  - This increases faithfulness
    - *Step 1: Extract relevant evidence.*
    - *Step 2: Provide the answer based only on that evidence.*
- Constrained output requires
  - Bullet-pointed answers
  - Direct quotes
  - Structured output format
  - More constraints reduces generative drift

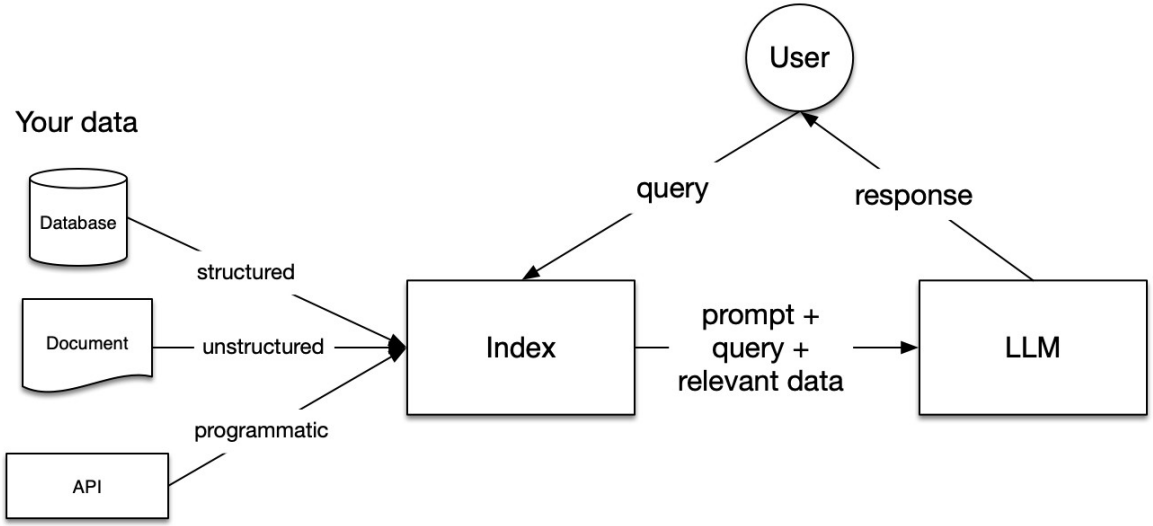


Image Credit: https://medium.com/@tselvaraj/the-role-of-prompt-engineering-in-rag-systems-f8aaf6f80327

© 2026 by Innovation In Software Corporation

Prompt Construction

- Handling conflicting documents
  - In enterprise corpora, documents may
    - Be outdated
    - Contradict each other
    - Reflect policy changes
  - The model must handle conflicts explicitly
- Strategies
  - Timestamp Preference
    - *"If documents conflict, prefer the most recent one"*
  - Conflict detection prompt
    - *'If documents contain conflicting information, explain the differences'*
  - Source Hierarchy
    - *'Prioritize: Official policy over informal documentation and Signed contracts over drafts'*
    - Metadata can guide ranking.

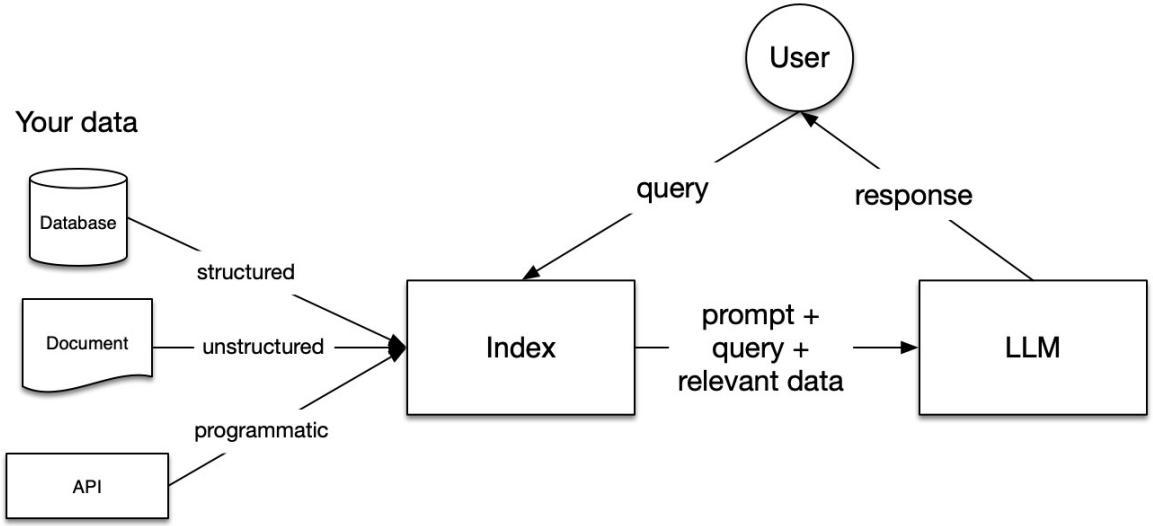


Image Credit: https://medium.com/@tselvaraj/the-role-of-prompt-engineering-in-rag-systems-f8aaf6f80327

© 2026 by Innovation In Software Corporation

Evaluating RAG Systems

- Evaluation determines whether a RAG system is
  - Trustworthy
  - Safe
  - Accurate
  - Production-ready
- RAG systems must be evaluated at multiple layers
  - Retrieval quality
  - Grounding quality
  - Generation quality
  - Robustness

© 2026 by Innovation In Software Corporation

Image Credit: https://medium.com/@sahin.samia/evaluating-rag-systems-metrics-and-best-practices-906a2c209bb5

Evaluating RAG Systems

- Precision and recall
  - If relevant documents are not retrieved, the LLM cannot answer correctly
  - Precision measures
    - Of the retrieved documents, how many are actually relevant
    - Precision= (Relevant Retrieved) / (Total Retrieved)
  - High precision
    - Less noise
    - Better context quality
    - Lower hallucination risk
  - Low precision
    - Distracting context
    - Increased confusion
    - Reduced answer clarity

© 2026 by Innovation In Software Corporation

Image Credit: https://medium.com/@sahin.samia/evaluating-rag-systems-metrics-and-best-practices-906a2c209bb5

Evaluating RAG Systems

- Precision and recall
  - Recall measures
    - Of all relevant documents in the corpus, how many were retrieved?
    - Recall=(Rel. Retrieved)/(Total Rel. in Corpus)
  - High recall
    - Greater coverage
    - Lower chance of missing critical evidence
  - Low recall
    - Missing key information
    - Increased hallucination risk
    - Incomplete answers

Image Credit: https://medium.com/@sahin.samia/evaluating-rag-systems-metrics-and-best-practices-906a2c209bb5

© 2026 by Innovation In Software Corporation

Evaluating RAG Systems

- Precision and recall
  - Trade-off
    - Increasing k improves recall
    - But may reduce precision.
  - Most production systems balance
    - Retrieval recall (broad coverage)
    - Re-ranking precision (final selection)

Image Credit: https://medium.com/@sahin.samia/evaluating-rag-systems-metrics-and-best-practices-906a2c209bb5

© 2026 by Innovation In Software Corporation

Evaluating RAG Systems

- Groundedness measures
  - Is the answer supported by retrieved documents?
  - This evaluates faithfulness, not just correctness.
  - An answer can be:
    - Correct but unsupported (risky)
    - Incorrect but confidently stated
    - Fully grounded in context (ideal)
  - In enterprise systems, grounding is often more important than fluency

© 2026 by Innovation In Software Corporation

Image Credit: https://medium.com/@sahin.samia/evaluating-rag-systems-metrics-and-best-practices-906a2c209bb5

Evaluating RAG Systems

- Groundedness measures
  - Evidence overlap checks whether statements in the answer
    - Appear in retrieved text
    - Can be directly traced to source chunks
  - Attribution scoring uses an LLM judge to verify
    - “Is this statement supported by the provided documents?”

© 2026 by Innovation In Software Corporation

Image Credit: https://medium.com/@sahin.samia/evaluating-rag-systems-metrics-and-best-practices-906a2c209bb5

Evaluating RAG Systems

- Hallucination detection
  - In RAG, hallucination often occurs when
    - Retrieval misses key evidence
    - Model extrapolates beyond context
    - Conflicting documents confuse reasoning
  - Types of Hallucination
    - Fabricated facts
    - Unsupported synthesis
    - Incorrect citation
    - Overconfident uncertainty

Image Credit: https://medium.com/@sahin.samia/evaluating-rag-systems-metrics-and-best-practices-906a2c209bb5

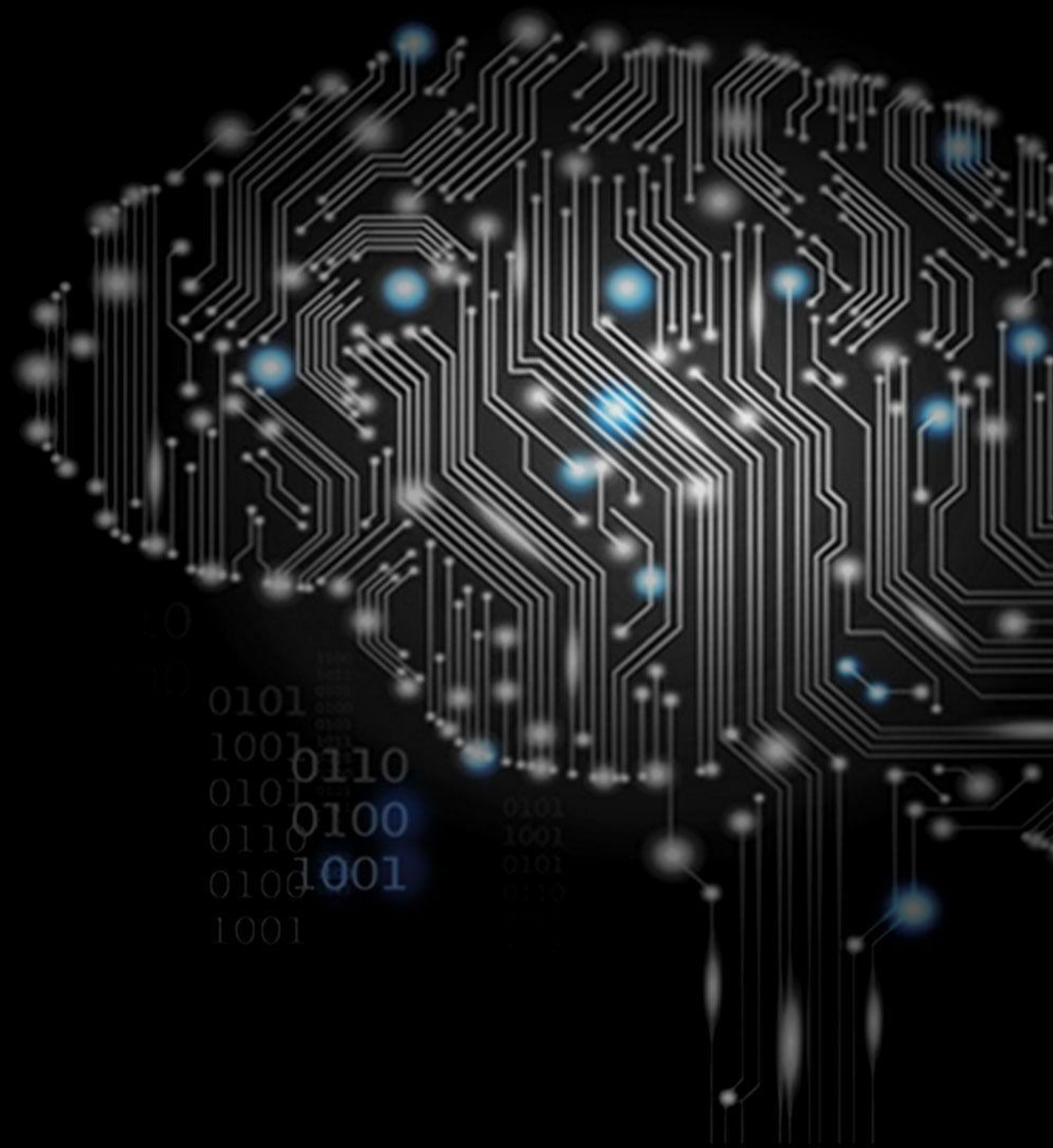
© 2026 by Innovation In Software Corporation

Evaluating RAG Systems

- Hallucination detection
  - In RAG, hallucination often occurs when
    - Retrieval misses key evidence
    - Model extrapolates beyond context
    - Conflicting documents confuse reasoning
  - Types of Hallucination
    - Fabricated facts
    - Unsupported synthesis
    - Incorrect citation
    - Overconfident uncertainty

Image Credit: https://medium.com/@sahin.samia/evaluating-rag-systems-metrics-and-best-practices-906a2c209bb5

© 2026 by Innovation In Software Corporation



Questions?